

**Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse
von Rich-Client-Webanwendungen mittels Tool-Reports,
Playwright-Interaktionen und Codeanalyse**

Bachelorarbeit

vorgelegt am 11.05.2026

Fakultät Wirtschaft und Gesundheit

Studiengang Wirtschaftsinformatik

Kurs WWI2023

von

JOEL YERAI MARTINEZ CAMPO

Betreuung in der Ausbildungsstätte:

DHBW Stuttgart:

BITBW

Sascha Alexander Ströbel

Ilko Hoffman

Betreuer der Bachelorarbeit

Unterschrift

Sperrvermerk

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungs- und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung des Dualen Partners vorliegt.

Ist das überhaupt notwendig bei meiner Arbeit? Ich habe es vorsichtshalber eingefügt.

Abstract der BA

Dies wird meine Bachelorarbeit

Muss noch ausgefüllt werden

Inhaltsverzeichnis

Sperrvermerk	I
Abkürzungsverzeichnis	V
Abbildungsverzeichnis	VI
Tabellenverzeichnis	VII
1 Einleitung	1
1.1 Motivation und Kontext	1
1.2 Problemstellung	2
1.3 Zielsetzung und Forschungsfragen	3
1.4 Aufbau der Arbeit	5
2 Theoretische Grundlagen	6
2.1 Barrierefreiheit von Webanwendungen	6
2.1.1 Begriffsdefinition und Relevanz	6
2.1.2 Web Content Accessibility Guidelines (WCAG): Aufbau, Prinzipien und Konformitätsstufen	8
2.1.3 Rechtlicher Rahmen	9
2.2 Automatisierte Barrierefreiheitsanalyse	10
2.2.1 Klassische Tool-Ansätze	10
2.2.2 Grenzen bestehender Tools	11
2.2.3 Taxonomie der Verstöße: syntaktisch, semantisch, Layout	12
2.3 Rich-Client-Webanwendungen und Testautomatisierung	14
2.3.1 Charakteristika und Abgrenzung	14
2.3.2 Herausforderungen für die Barrierefreiheitsprüfung	15
2.3.3 Playwright als Testautomatisierungswerkzeug	15
2.4 Künstliche Intelligenz (KI)-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen	16
2.4.1 Grundlagen: Large Language Models	16
2.4.2 Prompt-Engineering-Strategien	18
2.4.3 RAG und Fine-Tuning als Erweiterungsansätze	19
2.4.4 Datenschutz bei KI-gestützten Accessibility-Services	20
3 Stand der Forschung	22
3.1 Vorgehen der Literaturrecherche	22
3.2 Von frühen KI-Ansätzen zur Large Language Model (LLM)-basierten Erkennung	22
3.3 LLM-basierte Korrektur und Prompt-Engineering	23
3.4 Hybride Pipelines und Ensemble-Testing	23
3.5 Forschungslücken und Einordnung der vorliegenden Arbeit	24
4 Methodik	26
4.1 Wissenschaftliche Methode	26
4.2 Untersuchungsgegenstand und Scope-Abgrenzung	27
4.3 Evaluationsdesign und Kriterien	27

4.3.1	Evaluationslogik	28
4.3.2	Bewertungskriterien	28
4.3.3	Limitationen des Evaluationsdesigns	29
5	Analyse und Anforderungen	30
5.1	Problemkontext und Handlungsbedarf	30
5.2	Funktionale Anforderungen (FA)	31
5.3	Nicht-funktionale Anforderungen (NFA)	32
5.4	Modellauswahl: Ollama und qwen2.5-coder:7b	33
5.4.1	Ollama als Laufzeitumgebung	33
5.4.2	Begründung der Modellwahl	33
5.4.3	Zusammenfassung der Anforderungen	34
6	Konzeption des Assistenzsystems	36
6.1	Zielarchitektur und Datenpipeline	36
6.1.1	Analysequellen	36
6.1.2	Unified Finding Schema	37
6.2	Modell der Kontextualisierung und Priorisierung	38
6.2.1	Single-Prompt-Strategie	38
6.2.2	Token-Budget-Steuerung	39
6.2.3	Modellwahl	39
6.3	Ausgabeformat: entwicklernahe To-Do-Liste	39
7	Implementierung des Proof of Concept	41
7.1	Technische Umsetzung	41
7.2	Beispielabläufe und Ergebnisse am Untersuchungsobjekt	41
8	Evaluation & Diskussion	42
8.1	Ergebnisse entlang der Kriterien und Limitationen	42
8.2	Handlungsempfehlungen und Ausblick	42
9	Fazit	45
	Anhang	46
	Literaturverzeichnis	49

Abkürzungsverzeichnis

API	Application Programming Interface
BFSG	Barrierefreiheitsstärkungsgesetz
BITBW	IT Baden-Württemberg
BITV	Barrierefreie-Informationstechnik-Verordnung
CLI	Command Line Interface
DSR	Design Science Research
DOM	Document Object Model
DSGVO	Datenschutz-Grundverordnung
EAA	<i>European Accessibility Act</i>
FA	Funktionale Anforderungen
HTML	Hypertext Markup Language
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
LLM	Large Language Model
NFA	Nicht-funktionale Anforderungen
PoC	Proof of Concept
RAG	Retrieval-Augmented Generation
TF	Teilfragen
WAI	Web Accessibility Initiative
WCAG	Web Content Accessibility Guidelines
WHO	World Health Organization

Abbildungsverzeichnis

1	Prozessüberblick des geplanten Assistenzsystems	4
2	WCAG-Schichtenmodell	9
3	Taxonomie der Barrierefreiheitsverstöße	13
4	Framework Popularity über die letzten Jahre	14
5	Zero-Shot und Few-Shot Prompting im Vergleich	19
6	Komplexität von Home Pages (6 Jahre)	47

Tabellenverzeichnis

1	Erkennungsleistung von Accessibility-Tools	12
2	Konzeptmatrix (Hauptmatrix, hoch priorisierte Studien)	25
3	Zuordnung der DSRM-Phasen zu den Kapiteln der Arbeit.	26
4	Anforderungsübersicht: funktionale und nicht-funktionale Anforderungen mit Ursprung und Priorität.	35
5	Beispielhafte Accessibility-Verstöße und zugehörige Prüfquellen	42
6	Erkennungsrate der Violations im Proof of Concept	43
7	Bewertungsskala für die Empfehlungsqualität der LLM-Ausgaben	43
8	Bewertung der Empfehlungsqualität pro Violation	44
9	Konzeptmatrix (vollständige Übersicht)	48

1 Einleitung

Web-Barrierefreiheit ist eine zentrale Voraussetzung dafür, dass alle Menschen - unabhängig von individuellen Einschränkungen - gleichberechtigten Zugang zu digitalen Informationen und Dienstleistungen erhalten. In der Praxis zeigt sich jedoch eine erhebliche Diskrepanz zwischen den normativen Anforderungen und der tatsächlichen Umsetzung: Barrierefreiheit wird häufig spät berücksichtigt, bestehende Prüfwerkzeuge erfassen nur einen Bruchteil der tatsächlichen Barrieren und manuelle Korrekturen sind zeitaufwändig.¹ Diese Arbeit untersucht, wie ein LLM-basierter Ansatz diese Lücke schließen kann: durch die automatisierte Erkennung von Barrierefreiheitsproblemen in Webanwendungen und die Ableitung entwicklernaher Handlungsempfehlungen - lokal ausführbar und ohne den Transfer sensibler Daten an externe Dienste.²

1.1 Motivation und Kontext

Rund 1,3 Milliarden Menschen (16% der Weltbevölkerung) leben mit einer Form von Behinderung.³ Für diese Personengruppe ist der Zugang zu digitalen Angeboten keine bloße Komfortfrage, sondern eine grundlegende Voraussetzung für gesellschaftliche Teilhabe: ohne barrierefreie Webanwendungen bleiben ihnen wesentliche Lebensbereiche verschlossen - von der Kommunikation mit Behörden über den Zugang zu Bildung und Gesundheitsinformationen bis hin zur Teilnahme am wirtschaftlichen und gesellschaftlichen Leben. Das Recht auf barrierefreien Zugang zu Informations- und Kommunikationstechnologien ist völkerrechtlich in der UN-Behindertenrechtskonvention verankert und hat sich in den vergangenen Jahren zunehmend auch in verbindlichem nationalen und europäischen Recht niedergeschlagen.⁴

Auf europäischer Ebene verpflichtet der *European Accessibility Act* (EAA) die Mitgliedstaaten, Barrierefreiheitsanforderungen für digitale Produkte und Dienstleistungen durchzusetzen.⁵ In Deutschland wurde diese Richtlinie durch das Barrierefreiheitsstärkungsgesetz (BFSG) in nationales Recht überführt, das seit dem 28. Juni 2025 auch für weite Teile der Privatwirtschaft gilt.⁶ Ergänzt wird dieser Rahmen durch die Barrierefreie-Informationstechnik-Verordnung (BITV), die für Behörden und öffentliche Stellen bereits seit Jahren verbindliche Vorgaben formuliert.⁷

Trotz dieser eindeutigen Rechtslage ist die praktische Umsetzung barrierefreier Webanwendungen nach wie vor unzureichend. Die jährlich durchgeführte WebAIM-Million-Studie, die die Startseiten der meistbesuchten Webseiten weltweit analysiert, stellte 2025 fest, dass 94,8% der untersuchten Seiten nachweisbare Verstöße gegen die WCAG aufweisen.⁸ Im Vergleich mit den letzten

¹Vgl. Souza et al. 2024; Vgl. Abuaddous, Jali, Basir 2016, S. 175

²Vgl. Bassi et al. 2025, S. 297

³Vgl. World Health Organization 2026

⁴Vgl. United Nations 2006a

⁵Vgl. European Commission 2026

⁶Vgl. Deutscher Bundestag 2026

⁷Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025; Heinemann 2024, S. 281

⁸Vgl. WebAIM 2025

Jahren ist seit 2019 (97,8%) lediglich eine Verbesserung um 3 Prozentpunkte zu verzeichnen.⁹ Für die öffentliche Verwaltung ist diese Situation besonders brisant: während privatwirtschaftliche Anbieter erst seit Juni 2025 durch das BFSG verpflichtet sind, gelten für Behörden und öffentliche Stellen bereits seit Jahren verbindliche Barrierefreiheitsanforderungen nach der BITV - dennoch weisen auch zahlreiche Webangebote der öffentlichen Hand weiterhin erhebliche Mängel auf.¹⁰ Damit bleibt Barrierefreiheit in der Entwicklungspraxis trotz gesetzlicher Verpflichtung systematisch unterrepräsentiert.

Die IT Baden-Württemberg (BITBW) nimmt als zentrale IT-Dienstleisterin der Landesverwaltung Baden-Württemberg in diesem Kontext eine besondere Stellung ein.¹¹ Als Entwicklerin und Betreiberin öffentlicher Webanwendungen für Ministerien, Polizeibehörden und Kommunen ist sie gesetzlich zur Einhaltung der BITV und des BFSG verpflichtet.¹² Drei Faktoren machen die BITBW zu einem besonders geeigneten Praxiskontext für die vorliegende Arbeit: Erstens unterliegen ihre Anwendungen strengeren Barrierefreiheitsanforderungen als privatwirtschaftliche Produkte. Zweitens schließen die Datenschutzvorgaben der öffentlichen Verwaltung die Nutzung cloudbasierter LLM-Application Programming Interface (API)s weitgehend aus. Drittens erschweren gewachsene Systemlandschaften mit Legacy-Anteilen und begrenzten Entwicklungsressourcen die manuelle Barrierefreiheitsprüfung im erforderlichen Umfang. Sie bildet damit den praktischen Rahmen der vorliegenden Arbeit.

1.2 Problemstellung

Die mangelhafte Umsetzung von Barrierefreiheit in Softwareprojekten lässt sich nicht allein auf fehlende Motivation oder Ressourcen zurückführen. Ein wesentlicher Grund sind die Schwächen der verfügbaren Tools: automatisierte Barrierefreiheitsanalyse-Tools wie axe¹³, WAVE¹⁴ oder Lighthouse¹⁵ erkennen zwar syntaktische Verstöße, stoßen jedoch bei semantischen Problemen an ihre Grenzen. Ob ein vorhandener Alternativtext den Bildinhalt tatsächlich angemessen beschreibt, oder ob eine Schaltflächenbeschriftung für Screenreader-Nutzerinnen und -Nutzer verständlich ist, entzieht sich der regelbasierten Prüflöge dieser Tools.

Empirisch belegt wird diese Schwäche durch das Mutation-Testing-Framework **Mally**, das gezielt Barrierefreiheitsfehler in reale Webanwendungen injiziert und anschließend prüft, ob bestehende Accessibility-Tools diese künstlich eingebrachten Defekte erkennen.¹⁶ In einer systematischen Evaluation mit 25 gezielt eingebrachten Barrierefreiheitsproblemen erkannte kein einziges

⁹Vgl. WebAIM 2019

¹⁰Vgl. Heinemann 2024, S. 281

¹¹Vgl. BITBW 2021

¹²Vgl. Landesrecht BW 2015

¹³Vgl. Deque Systems 2026

¹⁴Vgl. WebAIM 2026a

¹⁵Vgl. Google 2025

¹⁶Vgl. Tafreshipour et al. 2024, S. 97

der sechs untersuchten Tools mehr als 50% der injizierten Fehler. Somit blieben im Durchschnitt nahezu die Hälfte aller Defekte unentdeckt.¹⁷

Eine zweite strukturelle Schwäche betrifft die Charakteristik moderner Webanwendungen. Rich-Client-Anwendungen auf Basis von Frameworks wie React erzeugen ihren Inhalt größtenteils clientseitig und verändern das Document Object Model (DOM) dynamisch in Abhängigkeit von Nutzerinteraktionen. Barrieren entstehen häufig erst in bestimmten Zuständen: wenn ein Modalfenster geöffnet wird, eine Fehlermeldung erscheint oder ein Dropdown-Menü aktiviert ist. Rein statische Analysen - oder solche, die ausschließlich den initialen Seitenladezustand prüfen - erfassen diese zustandsabhängigen Barrieren nicht.¹⁸

Jüngere Forschungsarbeiten zeigen, dass LLMs geeignet sind, die zuvor beschriebenen Grenzen regelbasierter Accessibility-Tools zu überwinden. Systeme wie **AccessGuru**¹⁹ und **GenA11y**²⁰ setzen das cloudbasierte LLM GPT-4o ein und erzielen bei der Erkennung und Korrektur von Barrierefreiheitsverstößen vielversprechende Ergebnisse. Beide Ansätze beschränken sich jedoch auf die Analyse statischen Hypertext Markup Language (HTML)s und berücksichtigen weder dynamische Zustandswechsel noch den Quellcode der geprüften Anwendung. Der Schritt von der Fehlererkennung zu einer entwicklernahen, auf den Quellcode bezogenen Handlungsempfehlung wird in der Literatur als offenes Problem benannt,²¹ bleibt aber bislang ungelöst. Eine vertiefte Analyse dieser und weiterer Arbeiten erfolgt in Kapitel 3.

Die vorliegende Forschungsliteratur lässt somit drei zentrale Defizite erkennen: es existiert bislang kein softwaregestütztes Tool, das Entwicklerinnen und Entwickler bei der Barrierefreiheitsanalyse unterstützt und dabei (a) die dynamischen Zustände einer Rich-Client-Anwendung durch automatisierte Browser-Interaktionen erfasst, (b) die Ergebnisse automatisierter Tool-Reports mit dem relevanten Quellcode verknüpft und (c) auf dieser Grundlage priorisierte, entwicklernaher Handlungsempfehlungen erzeugt - lokal ausführbar und ohne den Transfer sensibler Anwendungsdaten an externe Dienste.

1.3 Zielsetzung und Forschungsfragen

Ziel dieser Bachelorarbeit ist die Konzeption und prototypische Umsetzung einer lokal ausführbaren, kontextsensitiven Assistenzlösung zur Barrierefreiheitsanalyse von Rich-Client-Webanwendungen. Als Untersuchungsobjekt dient eine React-basierte Webanwendung aus dem Umfeld der BITBW. Der Proof of Concept soll drei Datenquellen in einer Datenpipeline zusammenführen: (1) strukturierte Violation-Reports aus axe-core, (2) Interaktionsdaten aus Playwright-gesteuerten Browser-Tests sowie (3) statische Quellcodeanalysen der geprüften React-Komponenten, etwa hinsichtlich fehlender `aria`-Attribute oder nicht-interaktiver Elemente mit `onClick`-Handlern.

¹⁷Vgl. Tafreshipour et al. 2024, S. 100

¹⁸Vgl. Bassi et al. 2025, S. 303

¹⁹Vgl. Fathallah, Hernández, Staab 2025

²⁰Vgl. He, Huq, Malek 2025

²¹Vgl. He, Huq, Malek 2025, FSE101:15–17

Auf dieser Grundlage werden priorisierte, entwicklernahe Handlungsempfehlungen abgeleitet und strukturiert aufbereitet. Abbildung 1 veranschaulicht diesen Prozess im Überblick.

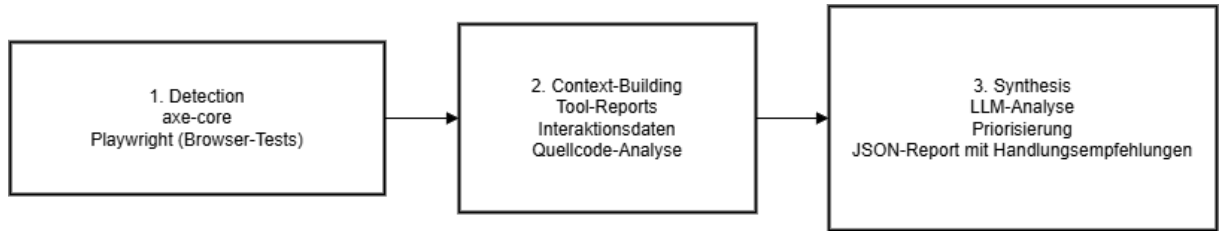


Abb. 1: Prozessüberblick: von den drei Datenquellen zur entwicklernahe Handlungsempfehlung²²

Diese Arbeit liefert drei konkrete Beiträge:

1. Einen strukturierten Überblick darüber, wie unterschiedliche Datenquellen der Barrierefreiheitsanalyse systematisch kombiniert und interpretiert werden können.
2. Einen Proof of Concept, der das Vorgehen exemplarisch demonstriert.
3. Eine qualitative Bewertung der Untersuchungsergebnisse anhand definierter Kriterien, die aus den Anforderungen und der einschlägigen Fachliteratur abgeleitet werden.

Die übergeordnete Forschungsfrage dieser Arbeit lautet:

Inwieweit kann ein kontextsensitives KI-Assistenzsystem, das Tool-Reports, Playwright-Interaktionsdaten und Quellcode kombiniert, Barrierefreiheitsverstöße in React-basierten Webanwendungen automatisiert erkennen und entwicklernahe Handlungsempfehlungen generieren?

Diese übergeordnete Frage wird durch zwei konkretisierende Teilfragen (TF) operationalisiert:

- TF1: Welche Typen von Barrierefreiheitsverstößen - syntaktisch, semantisch und layout-bezogen - lassen sich durch den gewählten Ansatz zuverlässig erkennen und wo liegen die methodischen Grenzen?
- TF2: Inwiefern verbessert die Kombination von Tool-Reports, Playwright-Interaktionsdaten und Quellcode die Qualität der generierten Handlungsempfehlungen gegenüber einer Analyse, die ausschließlich auf Tool-Reports und gerenderten DOM-Ausschnitten basiert?

TF1 adressiert dabei das Erkennungsproblem und erlaubt eine differenzierte Aussage über die Leistungsfähigkeit und Grenzen des Systems entlang der Verstoß-Taxonomie von Fathallah et al.²³ TF2 greift den in der Literatur identifizierten Mehrwert der Kontextualisierung durch Tool-Reports, Playwright-Interaktionsdaten und Quellcode auf und überprüft ihn im Rahmen eines qualitativen Ablationsvergleichs am gewählten Untersuchungsobjekt.

²²Eigene Abbildung

²³Vgl. Fathallah, Hernández, Staab 2025; Šmite et al. 2012, S. 114–115

1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in neun Kapitel. Im Anschluss an die Einleitung werden in Kapitel 2 die theoretischen Grundlagen erarbeitet: Barrierefreiheit im Web, die WCAG-Standards und der rechtliche Rahmen, die Charakteristika von Rich-Client-Webanwendungen sowie Grundlagen zu Large Language Models und Prompt-Engineering-Strategien.

Kapitel 3 gibt einen systematischen Überblick über den Stand der Forschung. Die relevanten Arbeiten werden thematisch gegliedert - nach LLM-basierter Erkennung, Code-Korrektur, hybriden Pipelines und Evaluation - und in einer Konzeptmatrix nach Webster und Watson zusammengefasst.²⁴ Das Kapitel schließt mit einer expliziten Einordnung des Beitrags der vorliegenden Arbeit.

Kapitel 4 legt das methodische Vorgehen dar. Es beschreibt die Wahl des Design-Science-Research-Paradigmas als wissenschaftliche Methode, definiert den Untersuchungsgegenstand und entwirft das Evaluationsdesign.²⁵ Kapitel 5 analysiert den Problemkontext und leitet daraus funktionale und nicht-funktionale Anforderungen ab, einschließlich der Begründung der Modellauswahl.

Auf Basis dieser Anforderungen wird in Kapitel ?? das Assistenzsystem konzipiert: Systemarchitektur, Datenpipeline, Prompt-Strategie und Ausgabeformat werden begründet entworfen. Kapitel 7 beschreibt die prototypische Implementierung des Proof of Concept und illustriert die Systemfunktionalität anhand von Beispielabläufen für syntaktische, semantische und dynamisch erzeugte Barrierefreiheitsprobleme.

Kapitel 8 präsentiert und diskutiert die Evaluationsergebnisse, beantwortet die Forschungsfragen und reflektiert die Limitationen des Ansatzes. Das abschließende Kapitel 9 fasst die Kernaussagen zusammen, ordnet den Beitrag der Arbeit in die Forschungslandschaft ein und formuliert Empfehlungen für weiterführende Untersuchungen.

²⁴Vgl. Webster, Watson 2002

²⁵Vgl. Hevner et al. 2004, S. 75

2 Theoretische Grundlagen

Das folgende Kapitel legt die theoretischen Grundlagen für die Einführung des Proof of Concepts dar. Es werden Begriffe und Konzepte erläutert, die für das Verständnis der Problemstellung und des entwickelten Ansatzes erforderlich sind. Zunächst wird Barrierefreiheit im Web als Untersuchungsgegenstand definiert (Abschnitt 2.1), anschließend der Stand automatisierter Barrierefreiheitstools sowie ihre Möglichkeiten und Grenzen (Abschnitt 2.2). Darauf aufbauend werden dann die besonderen Herausforderungen von Rich-Client-Anwendungen und das Werkzeug Playwright eingeführt (Abschnitt 2.3). Das Kapitel schließt mit einer Einführung in LLM, Modelle und relevante Prompting Strategien sowie auch Datenschutzbetrachtungen (Abschnitt 2.4).

2.1 Barrierefreiheit von Webanwendungen

2.1.1 Begriffsdefinition und Relevanz

Der Begriff *Web Accessibility*, im Deutschen Web-Barrierefreiheit, bezeichnet nach der Definition der Web Accessibility Initiative (WAI) die Eigenschaft von Webangeboten, von allen Menschen unabhängig von körperlichen, motorischen oder sensorischen Einschränkungen sowie individuellen Fähigkeiten wahrgenommen werden zu können.²⁶ Web Accessibility geht jedoch über die reine Wahrnehmbarkeit hinaus und umfasst auch die Bedienbarkeit sowie Nutzbarkeit von Webangeboten.²⁷ Laut Angaben der World Health Organization (WHO) wird geschätzt, dass rund 16% der Weltbevölkerung mit einer Form von Behinderung leben.²⁸ Diese Zahl umfasst ein breites Spektrum an Einschränkungen, von denen nicht alle unmittelbar die Nutzung digitaler Angebote betreffen - eine Person im Rollstuhl ohne weitere sensorische oder motorische Einschränkungen der oberen Extremitäten ist beispielsweise bei der Webnutzung nicht zwingend eingeschränkt. Entscheidend für die Web-Barrierefreiheit sind vor allem visuelle, motorische und kognitive Beeinträchtigungen, die die Wahrnehmung oder Bedienung digitaler Oberflächen erschweren. Darüber hinaus profitieren auch Menschen ohne dauerhafte Behinderung von barrierefreier Gestaltung: temporäre Einschränkungen wie ein gebrochener Arm oder situationsbedingte Faktoren wie gleißendes Sonnenlicht auf dem Bildschirm können die Nutzung digitaler Angebote vorübergehend erheblich erschweren.²⁹ Damit stellt Barrierefreiheit im Web nicht nur eine technische Herausforderung dar, sondern auch eine zentrale gesellschaftliche Aufgabe zur Gewährleistung digitaler Teilhabe. Das Recht auf barrierefreien Zugang zu Informations- und Kommunikationstechnologien ist seit 2006 in Art. 9 der UN-Behindertenrechtskonvention völkerrechtlich verankert.³⁰

²⁶Vgl. Abuaddous, Jali, Basir 2016, S. 172

²⁷Vgl. World Wide Web Consortium (W3C) 2026b

²⁸Vgl. World Health Organization 2026

²⁹Vgl. Organization 2026

³⁰Vgl. United Nations 2006b, S. 9

In der Forschungsliteratur werden vier wesentliche Behinderungsformen unterschieden, die jeweils spezifische Anforderungen an die Gestaltung barrierefreier Webangebote stellen.³¹ Für die vorliegende Arbeit sind insbesondere zwei Kategorien relevant - visuelle Beeinträchtigungen und motorische Einschränkungen -, da sie den Großteil der automatisiert prüfbaren WCAG-Verstöße betreffen:

Visuelle Beeinträchtigungen umfassen vollständige oder partielle Blindheit sowie Farbfehlsichtigkeit.³² Betroffene Personen nutzen häufig Screenreader-Software (z. B. JAWS, NVDA, VoiceOver), die auf korrekte semantische HTML-Auszeichnung angewiesen ist.³³ Fehlende Alternativtexte, unzureichende Farbkontraste und fehlende **aria**-Labels bilden die häufigsten Verstöße in dieser Kategorie³⁴ und zugleich den primären Erkennungsbereich des in dieser Arbeit entwickelten Systems.

Motorische Einschränkungen betreffen Personen, die konventionelle Eingabegeräte wie Maus oder Touchscreen nicht präzise bedienen können.³⁵ Sie sind auf Tastaturnavigation, Sprachsteuerung oder spezielle assistive Eingabegeräte angewiesen.³⁶ Eine vollständige Bedienbarkeit über die Tastatur ist daher eine zentrale Voraussetzung für barrierefreie Nutzung.³⁷ Für das vorliegende System ist diese Kategorie besonders relevant, da Tastaturfallen und fehlende Fokussteuerung in (React-)Anwendungen häufig erst durch interaktionsbasierte Tests - etwa mittels Playwright - aufgedeckt werden können.³⁸

Daneben adressiert die WCAG auch **auditive Beeinträchtigungen** (Untertitel, Transkriptionen)³⁹ sowie **kognitive und neurologische Beeinträchtigungen** (klare Sprache, konsistente Strukturen, reduzierte visuelle Komplexität).⁴⁰ Beide Kategorien erfordern jedoch überwiegend menschliches Urteilsvermögen und entziehen sich weitgehend der automatisierten Prüfung.⁴¹ Sie liegen daher außerhalb des Scopes des vorliegenden Proof of Concept (PoC).

Assistive Technologien - darunter Screenreader, Bildschirmvergrößerungssoftware und alternative Eingabegeräte - bilden die technische Voraussetzung für die Nutzung digitaler Angebote durch die genannten Personengruppen.⁴² Ihre Wirksamkeit hängt jedoch unmittelbar davon ab, dass Webanwendungen klar definierte strukturelle und semantische Anforderungen erfüllen.⁴³ Die Nichterfüllung dieser Anforderungen führt nicht nur zu einer Beeinträchtigung der betroffenen Personen, sondern kann auch wirtschaftliche und rechtliche Konsequenzen nach sich ziehen.⁴⁴

³¹Vgl. Lazar, Feng, Hochheiser 2017, S. 493–494

³²Vgl. Gubbi Mohanbabu, Pavel 2024

³³Vgl. Morris et al. 2016, S. 5508

³⁴Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 1.1.1

³⁵Vgl. Pérez et al. 2020, S. 110381–110382

³⁶Vgl. Kouroupetroglou 2013, S. 208

³⁷Vgl. WebAIM 2026b

³⁸Vgl. Chiou, Alotaibi, Halfond 2021, S. 855

³⁹Vgl. Harrenstien 2009

⁴⁰Vgl. World Wide Web Consortium (W3C) 2023

⁴¹Vgl. Abou-Zahra, Brewer, Cooper 2018

⁴²Vgl. World Wide Web Consortium (W3C) 2026b; Abou-Zahra, Brewer, Cooper 2018

⁴³Vgl. Morrison et al. 2017, S. 82

⁴⁴Vgl. Krok 2024, S. 864–865

2.1.2 WCAG: Aufbau, Prinzipien und Konformitätsstufen

Die WCAG sind der internationale Standard für barrierefreie Webinhalte und werden vom World Wide Web Consortium (W3C) im Rahmen der WAI herausgegeben. Die derzeit maßgebliche Referenzversion ist WCAG 2.1 aus dem Jahr 2018, die seit Oktober 2023 durch die WCAG 2.2 ergänzt wird.⁴⁵ Die Version 2.2 führt neun zusätzliche Erfolgskriterien ein und berücksichtigt insbesondere kognitive Einschränkungen und mobile Nutzung.⁴⁶ Zu diesen Kriterien gehören konsistentere Fokusindikatoren (2.4.11–2.4.13) und die Vermeidung von Drag-and-Drop-Pflichtinteraktionen (WCAG 2.5.7).⁴⁷

Die WCAG ist in vier Schichten gegliedert: Prinzipien, Richtlinien, Erfolgskriterien und unterstützende Techniken. Auf der obersten Ebene stehen vier grundlegende Prinzipien, zusammengefasst im Akronym **POUR**:⁴⁸

- **Perceivable (Wahrnehmbar):** Alle Informationen und Benutzerschnittstellen müssen für alle Nutzer und Nutzerinnen wahrnehmbar sein.⁴⁹ Typische Anforderungen sind Alternativtexte für Bilder (Erfolgskriterium 1.1.1) und ausreichende Farbkontraste (Erfolgskriterium 1.4.3).⁵⁰
- **Operable (Bedienbar):** Alle Funktionen der Benutzeroberfläche müssen über die Tastatur erreichbar sein.⁵¹ Zeitlich beschränkte Inhalte, die nach einer vordefinierten Zeit verschwinden, müssen auch verhindert werden. Auch Inhalte, die Anfälle auslösen können, müssen vermieden werden.⁵²
- **Understandable (Verständlichkeit):** Inhalte und Bedienelemente müssen so verständlich gestaltet sein, dass die verwendete Sprache angegeben ist und Fehlermeldungen aussagekräftig sowie beschreibend formuliert sind.⁵³
- **Robust (Robust):** Inhalte müssen so implementiert sein, dass sie von einer breiten Palette von aktuellen und zukünftigen assistiver Technologien interpretiert und ausgeführt werden können.⁵⁴

Die 13 Richtlinien unterhalb der Prinzipien werden durch insgesamt 78 testbare Erfolgskriterien operationalisiert (WCAG 2.2).⁵⁵ Jedes Erfolgskriterium ist einer von drei Konformitätsstufen zugeordnet.⁵⁶ Stufe A umfasst grundlegende Anforderungen, deren Nichteinhaltung bestimmte

⁴⁵Vgl. World Wide Web Consortium (W3C) 2018; World Wide Web Consortium (W3C) 2023

⁴⁶Vgl. Purwandari et al. 2025, S. 118; Vgl. Souza et al. 2024

⁴⁷Vgl. World Wide Web Consortium (W3C) 2023

⁴⁸Vgl. Souza et al. 2024

⁴⁹Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 1

⁵⁰Vgl. Souza et al. 2024

⁵¹Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 2

⁵²Vgl. Souza et al. 2024

⁵³Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 3

⁵⁴Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 4

⁵⁵Vgl. World Wide Web Consortium (W3C) 2023

⁵⁶Vgl. Fathallah, Hernández, Staab 2025

Nutzergruppen vollständig ausschließt. Stufe AA ist der in Europa gesetzlich geforderte Mindeststandard für öffentliche Stellen und weitere Teile der Privatwirtschaft.⁵⁷ Stufe AAA beschreibt Anforderungen, die nicht für alle Inhalte generell erfüllbar sind. Die vorliegende Arbeit fokussiert sich entsprechend dem gesetzlichen Mindeststandard auf Stufe A und AA.

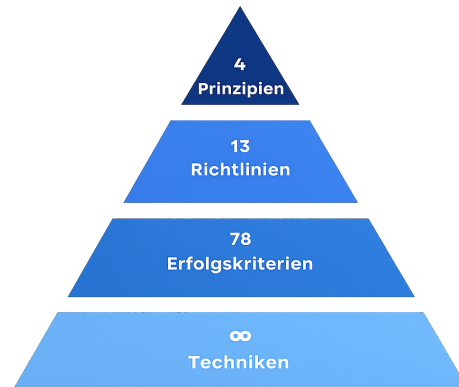


Abb. 2: WCAG-Schichtenmodell⁵⁸

Die praktische Umsetzung dieser Standards ist trotz ihrer langen Existenz nach wie vor unzureichend.⁵⁹ Die WebAIM-Million-Studie 2025 identifizierte im Durchschnitt 56,8 WCAG-Fehler pro Seite. Die häufigsten Verstößkategorien betrafen fehlende Alternativtexte (54,5%), unzureichende Farbkontraste (80,6%), fehlende Labels (47,4%), leere Links (44,6%) und fehlende Sprachangaben im HTML (17,1%).⁶⁰

2.1.3 Rechtlicher Rahmen

Auf europäischer Ebene bildet der *European Accessibility Act* (EAA) den übergeordneten rechtlichen Rahmen.⁶¹ Er verpflichtet Mitgliedstaaten, einheitliche und barrierefreiheitskonforme Inhalte und Webangebote für die digitalen Produkte und Dienstleistungen durchzusetzen.

In Deutschland wurde der EAA durch das Barrierefreiheitsstärkungsgesetz (BFSG) in nationales Recht überführt.⁶² Das BFSG ist seit dem 28. Juni 2025 für privatwirtschaftliche Anbieter verbindlich. Unternehmen mit weniger als zehn Beschäftigten und einem Jahresumsatz unter zwei Millionen Euro sind von bestimmten Pflichten ausgenommen.⁶³

Für öffentliche Stellen des Bundes gilt in Deutschland bereits seit 2011 die Barrierefreie-Informationstechnik-Verordnung (BITV) 2.0, die Behörden verpflichtet, ihre Webangebote nach WCAG 2.1 auf mindestens Konformitätsstufe AA zu gestalten und zu dokumentieren.⁶⁴ Die BITV 2.0

⁵⁷Vgl. Kerkmann, Lewandowski 2015, S. 40, 195 ff.

⁵⁸Eigene Abbildung

⁵⁹Vgl. Bassi et al. 2025, S. 297

⁶⁰Vgl. WebAIM 2025

⁶¹Vgl. European Commission 2026

⁶²Vgl. Deutscher Bundestag 2026

⁶³Vgl. Bundesamt der Justiz und für Verbraucherschutz 2026, §1 (Anwendungsbereich) und §3 (Barrierefreiheit)

⁶⁴Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

verweist in ihrer Anlage 1 unmittelbar auf die WCAG-Erfolgskriterien und ist somit dynamisch mit deren Weiterentwicklung verknüpft.

Diese rechtlichen Anforderungen sind für alle betroffenen Organisationen verbindlich - öffentliche Stellen ebenso wie privatwirtschaftliche Anbieter im Geltungsbereich des BFGG. Die IT Baden-Württemberg (BITBW) als zentrale IT-Dienstleisterin der Landesverwaltung Baden-Württembergs ist davon in besonderem Maße betroffen, da sämtliche von ihr entwickelten oder betriebenen Anwendungen den Anforderungen entsprechen müssen.⁶⁵ Dies begründet das praktische Interesse an effizienten, automatisierten Methoden der Barrierefreiheitsüberprüfung, wie sie in dieser Arbeit untersucht werden.

2.2 Automatisierte Barrierefreiheitsanalyse

Trotz der klar definierten Richtlinien der WCAG zeigt die Praxis, dass Barrierefreiheit in der Webentwicklung häufig unzureichend umgesetzt wird. Ein zentraler Grund dafür liegt in den Entwicklungsprozessen moderner Webanwendungen. In vielen Softwareprojekten wird Barrierefreiheit nicht von Beginn an als integraler Bestandteil der Entwicklung berücksichtigt, sondern erst in späten Testphasen oder gar nicht adressiert. Hinzu kommt, dass viele Entwicklerinnen und Entwickler nur begrenztes Wissen über die WCAG-Anforderungen besitzen und Accessibility-Aspekte nicht automatisch berücksichtigt werden.⁶⁶ Auch der zunehmende Einsatz komplexer Architekturen und Frameworks erschwert die korrekte Umsetzung barrierefreier Interaktionen. In Kombination mit Zeitdruck und fehlenden automatisierten Prüfverfahren führt dies dazu, dass Accessibility-Probleme häufig erst spät erkannt oder vollständig übersehen werden.⁶⁷

2.2.1 Klassische Tool-Ansätze

Für die automatisierte Überprüfung von Webanwendungen auf Barrierefreiheit gibt es eine stetig wachsende Anzahl an Werkzeugen. Die WAI führt in ihrer aktuellen Evaluation-Tool-Liste über 160 Werkzeuge⁶⁸, von denen 84 explizit die WCAG 2.1 adressieren.⁶⁹ Alle gängigen Werkzeuge arbeiten regelbasiert, d. h. sie prüfen den DOM einer geladenen Seite anhand eines vordefinierten Regelkatalogs gegen bekannte Verstoßmuster.⁷⁰ Das Document Object Model (DOM) ist eine vom Browser erzeugte, baumartige Objektstruktur, die den HTML-Code einer Webseite als hierarchische Struktur aus Elementknoten repräsentiert und über Programmierschnittstellen von Skripten und Analysewerkzeugen zugänglich macht.⁷¹

Zu den meistgenutzten automatisierten Barrierefreiheitstools gehören:

⁶⁵ Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

⁶⁶ Vgl. Bassi et al. 2025, S. 298

⁶⁷ Vgl. Mowar 2024, S. 123

⁶⁸ Vgl. World Wide Web Consortium (W3C) 2026a

⁶⁹ Vgl. Delnevo, Andruccioli, Mirri 2024

⁷⁰ Vgl. Kodithuwakku, Wickramaarachchi 2025; Manca et al. 2023, 3:9

⁷¹ Vgl. Robie 2026

- **axe-core**, entwickelt von Deque Systems, hat sich als Standardlösung für die automatische Integration von Accessibility-Tests in Entwicklungspipelines etabliert.⁷²
- **WAVE** von WebAIM bietet visuelle Browser-Extensions, die Fehler direkt in der gerenderten Seitenansicht annotiert.⁷³
- **Lighthouse** ist in die Chrome-DevTools integriert und kombiniert Accessibility-Testing mit SEO-Metriken und Performanceanalysen.⁷⁴
- **IBM Equal Access Checker** und **ARC Toolkit** (TPGi) runden das Spektrum der verbreiteten Werkzeuge ab.

Der typische Output solcher Werkzeuge ist ein strukturierter Violation Report, bei dem jedem erkannten Verstoß eine Bezeichnung, Beschreibung und Schweregrad zugewiesen wird (Beispielsweise bei axe-core: minor, moderate, serious, critical), sowie falls möglich auch ein HTML-Ausschnitt des betroffenen Elements ausgegeben.⁷⁵ Dieser standardisierte Output bildet in der vorliegenden Arbeit die Grundlage für die erste Stufe der Datenpipeline.

2.2.2 Grenzen bestehender Tools

Trotz ihrer weiten Verbreitung weisen regelbasierte Accessibility-Tools fundamentale Grenzen auf. Diese lassen sich in drei Kategorien einteilen: begrenzte Regelabdeckung, semantische Blindheit und fehlende Dynamik.

Die Regelabdeckung der Werkzeuge ist erheblich geringer als oftmals angenommen. Kodithuwakku und Wickramarachchi zeigen, dass selbst die leistungsfähigsten untersuchten Tools lediglich 21% aller WCAG-Tests automatisiert abdecken können (vgl. Tabelle 1).⁷⁶ Für Tastaturzugänglichkeit und auditive Barrieren lag die Erkennungsrate in ihrer Studie bei null Prozent. Ein zusätzliches Ergebnis der Studie war die hohe Rate an *False Positives*, die eine manuelle Nachprüfung erfordern.⁷⁷ Bassi et al. kommen zu dem Ergebnis, dass das beste verfügbare Einzeltool (**ARC Toolkit**) nur 26% der WCAG-Tests abdeckt.⁷⁸ Die Kombination mehrerer Tools in einem Ensemble verbessert die Abdeckung, beseitigt das strukturelle Problem jedoch nicht vollständig. Pool zeigt, dass selbst ein Ensemble aus acht gleichzeitig eingesetzten Tools, darunter auch **axe-core**, **WAVE**, **Qualweb** usw., erhebliche Lücken hinterlässt.⁷⁹ Insgesamt stellte sich in der Studie heraus, dass jedes einzelne Tool mindestens sieben Violations exklusiv erkennt, die von keinem anderen Tool gefunden werden. Problematisch ist dabei auch, dass die meisten Tools

⁷²Vgl. Deque Systems 2026

⁷³Vgl. WebAIM 2026a

⁷⁴Vgl. Google 2025

⁷⁵Vgl. Kodithuwakku, Wickramarachchi 2025; Deque Systems 2026

⁷⁶Vgl. Kodithuwakku, Wickramarachchi 2025

⁷⁷Vgl. Kodithuwakku, Wickramarachchi 2025

⁷⁸Vgl. Bassi et al. 2025, S. 298

⁷⁹Vgl. Pool 2023

ihre eigene Abdeckung nicht kommunizieren: Manca et al. stellen fest, dass von elf untersuchten Werkzeugen lediglich drei explizit auf ihre Erkennungslücken hinweisen.⁸⁰

Tool	True Positives	False Positives	CER (%)	ICR (%)
Lighthouse	17	16	52.94	15.92
WAVE	16	12	35.29	14.65
SiteImprove	23	18	47.06	19.75
axe	15	10	52.94	15.29
Accessibility Insights	24	19	47.06	15.29
A11y	6	8	23.53	3.82
Includia Accessibility Checker	33	31	41.18	21.02

Tab. 1: Vergleich der Erkennungsleistung verschiedener Accessibility-Prüfwerkzeuge.⁸¹ CER beschreibt den Anteil korrekt erkannter Verstöße im Verhältnis zu allen gemeldeten Verstößen, während ICR den Anteil tatsächlich vorhandener Accessibility-Probleme misst, die vom jeweiligen Tool erkannt wurden.

Das gravierendste Defizit liegt in der **semantischen Blindheit** der Werkzeuge.⁸² Regelbasierte Tools können lediglich prüfen, ob ein Accessibility-Attribut vorhanden ist, nicht aber, ob der Inhalt dem richtigen Zweck dient. Ein Bild mit dem Alternativtext *image* oder *IMG_0042.jpg* wird von allen Tools als regelkonform eingestuft, obwohl dieser Text für Screenreadernutzer und Nutzerinnen keinen Mehrwert schafft. **Ma11y**, ein Mutation-Testing-Framework für Accessibility, hat dieses Problem empirisch quantifiziert: bei der systematischen Injektion von 25 gezielten Defekten in reale Webanwendungen erkannten die sechs ausgewählten Tools semantische Verstöße nur zu 26%.⁸³ Die Quote der erkannten syntaktischen Verstöße dagegen lag bei 54%.⁸⁴ Im Durchschnitt blieben also nahezu die Hälfte aller injizierten Defekte unentdeckt.⁸⁵

Ein weiteres strukturelles Problem ist die **fehlende Dynamik** der Analyse. Viele Tools analysieren ausschließlich den initial geladenen DOM-Zustand einer Seite.⁸⁶ Barrieren, die erst durch Nutzerinteraktionen entstehen, wie ein Hilfs- oder Modalfenster, das geöffnet wird; ein Dropdown-Menü, um eine Kategorie auszuwählen; oder das Erscheinen einer Fehlermeldung, bleiben dabei unsichtbar. Hinzu kommt, dass alle Violation Reports generischer Natur sind. Sie beschreiben das Problem auf der Ebene des gerenderten DOM, ohne Bezug zur Quellcodestruktur der geprüften Anwendung herzustellen.⁸⁷

2.2.3 Taxonomie der Verstöße: syntaktisch, semantisch, Layout

Um Barrierefreiheitsverstöße systematisch analysieren und gezielt adressieren zu können, wird in dieser Arbeit eine dreiteilige Taxonomie zugrunde gelegt, die auf Fathallah et al. zurückgeht

⁸⁰Vgl. Manca et al. 2023, S. 4

⁸²Vgl. Fathallah, Hernández, Staab 2025

⁸³Vgl. Tafreshipour et al. 2024, S. 108

⁸⁴Vgl. Tafreshipour et al. 2024, S. 108

⁸⁵Vgl. Tafreshipour et al. 2024, S. 108

⁸⁶Vgl. Fathallah, Hernández, Staab 2025

⁸⁷Vgl. Delnevo, Andruccioli, Mirri 2024

(vgl. Abbildung 3).⁸⁸

- **Syntaktische Verstöße** bezeichnen Fälle, in denen ein erforderliches Accessibility-Element vollständig fehlt oder formal falsch gebildet ist.⁸⁹ Typische Beispiele sind ein fehlendes `alt`-Attribut an einem `<img>`-Element, ein fehlender `aria-label` an einer Schaltfläche ohne sichtbaren Text oder `<table>` ohne `<th>`-Elemente. Diese Kategorie ist für automatisierte regelbasierte Tools am besten prüfbar, da die Verletzung rein struktureller Natur ist.⁹⁰
- **Semantische Verstöße** liegen vor, wenn Accessibility-Attribute zwar vorhanden sind, ihren Zweck aber inhaltlich verfehlen.⁹¹ Ein Bild mit dem Alternativtext *Bild*, ein Formularelement mit der Beschriftung *Eingabe* oder ein Link mit dem Text *hier klicken* sind typische Beispiele. Da die Bewertung solcher Verstöße Sprachverständnis und Kontextwissen erfordert, sind diese viel schwieriger von Tools zu entdecken und bilden damit das primäre Anwendungsfeld für LLM-basierte Analysen.⁹²
- **Layout-Verstöße** betreffen visuelle Barrieren. Zu dieser Kategorie zählen insbesondere unzureichende Farbkontraste⁹³, fehlende oder schwer erkennbare Fokusindikatoren⁹⁴ sowie eine eingeschränkte Zoomfähigkeit. Einige dieser Verstöße können algorithmisch geprüft werden; andere erfordern ein kontextuelles visuelles Urteil.

Taxonomie der Barrierefreiheitsverstöße

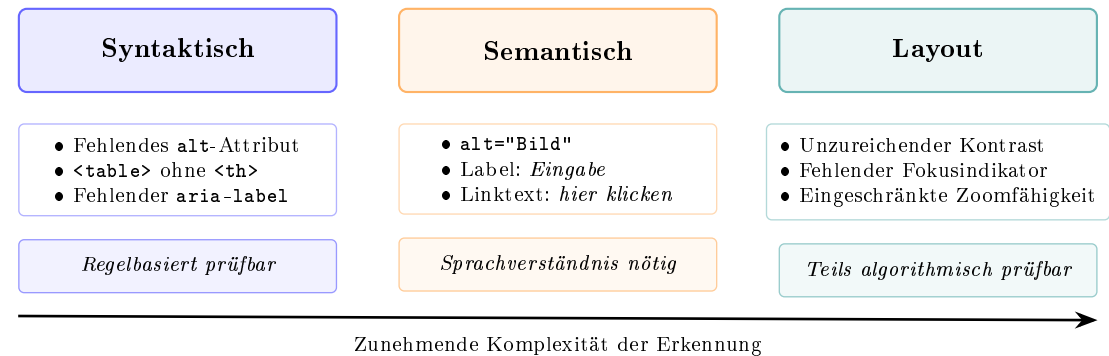


Abb. 3: Taxonomie der Barrierefreiheitsverstöße nach Fathallah et al. mit Beispielen und Erkennungscharakteristik je Kategorie⁹⁵

⁸⁸Vgl. Fathallah, Hernández, Staab 2025

⁸⁹Vgl. Tafreshipour et al. 2024, S. 102

⁹⁰Vgl. Flanagan 2020, S. 569 ff.

⁹¹Vgl. World Wide Web Consortium (W3C) 2023

⁹²Vgl. He, Huq, Malek 2025, FSE101:2

⁹³Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 1.4.3

⁹⁴Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 2.4.7–2.4.13

⁹⁵Eigene Abbildung, inspiriert aus Fathallah, Hernández, Staab 2025

2.3 Rich-Client-Webanwendungen und Testautomatisierung

2.3.1 Charakteristika und Abgrenzung

Der Begriff **Rich-Client-Webanwendung** bezeichnet Webanwendungen, bei denen ein Großteil der Anwendungslogik und Darstellungssteuerung clientseitig im Browser ausgeführt wird.⁹⁶ Im Gegensatz zu klassischen server-seitig gerenderten Anwendungen werden bei einer Single-Page Application (SPA) nach dem initialen Laden des HTML-Rahmens ausschließlich Daten zwischen Client und Server ausgetauscht, wobei die Darstellung durch clientseitige Skriptsprachen wie JavaScript oder TypeScript direkt im Browser durch Manipulation des DOM gesteuert wird.⁹⁷ Die zunehmende Komplexität solcher Anwendungen ist in Anhang 1 veranschaulicht.

React, das von Meta entwickelte JavaScript-Framework, ist der dominante Ansatz für die Entwicklung solcher Anwendungen. Laut Stack Overflow Developer Survey 2024 wird React von 44,7% aller befragten Entwicklerinnen und Entwickler eingesetzt.⁹⁸

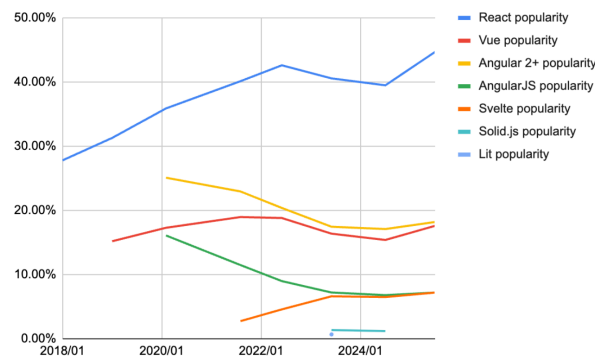


Abb. 4: Framework Popularity über die letzten Jahre⁹⁹

Für die Barrierefreiheitsanalyse sind zwei Architekturmerkmale von React besonders relevant: erstens werden Anwendungslogik und Darstellung in wiederverwendbaren **Komponenten** gekapselt, deren Kommunikation über **Props** (unveränderliche Eingabeparameter) und internen **State** (veränderlicher Zustand) erfolgt.¹⁰⁰ Zweitens wird die Darstellung der Komponenten mithilfe der Syntaxerweiterung JSX (bzw. TSX bei Verwendung von TypeScript) definiert, die HTML-ähnliche Strukturen direkt in JavaScript- oder TypeScript-Code einbettet.¹⁰¹

⁹⁶Vgl. Meta Open Source 2026a

⁹⁷Vgl. Meta Open Source 2026b

⁹⁸Vgl. Stack Overflow 2025

⁹⁹Entnommen aus Krotz 2026

¹⁰⁰Vgl. Meta Open Source 2026a

¹⁰¹Vgl. Bai 2022

2.3.2 Herausforderungen für die Barrierefreiheitsprüfung

Komponentenbasierte Frameworks wie React, Vue oder Angular teilen bestimmte Architekturmerkmale, die die Barrierefreiheitsanalyse grundsätzlich erschweren.¹⁰² Da die vorliegende Arbeit eine React-Anwendung als Untersuchungsgegenstand verwendet, werden die folgenden Herausforderungen anhand von React konkretisiert; sie gelten in ähnlicher Form jedoch auch für andere komponentenbasierte Frameworks.

Erstens entstehen viele Barrierefreiheitsprobleme **zustandsabhängig**, wie bereits in Kapitel 2.2.2 erläutert. Ein Modalfenster, das nach einem Klick geöffnet wird und den Tastaturfokus nicht korrekt verwaltet; eine Fehlermeldung, die nach dem Absenden eines Formulars erscheint und nicht durch eine ARIA-Live-Region programmatisch angekündigt wird; ein Dropdown-Menü, das für Tastaturnutzer nicht navigierbar ist - all diese Probleme sind im initialen Seitenzustand nicht vorhanden und damit für statische Analysewerkzeuge unsichtbar.¹⁰³ In React wird diese Problematik durch das State-Management verstärkt: Zustandsänderungen lösen ein erneutes Rendering einzelner Komponenten aus, wobei sich die Accessibility-Eigenschaften des resultierenden DOM je nach State unterscheiden können.

Zweitens besteht eine **Diskrepanz zwischen Quellcode und gerendertem DOM**. In React werden Benutzeroberflächen in JSX/TSX definiert; der im Browser gerenderte DOM ist das Ergebnis der React-Laufzeitumgebung, die diese Definitionen in DOM-Elemente übersetzt. Ein Accessibility-Tool analysiert den gerenderten DOM, nicht aber den Quellcode, aus dem er hervorging.¹⁰⁴ Für Entwicklerinnen und Entwickler ist jedoch der Quellcode der relevante Interventionspunkt.¹⁰⁵ Ein konkretes Beispiel: eine React-Komponente `IconButton` rendert im DOM ein `<button>`-Element mit einem `<svg>`-Icon, jedoch ohne sichtbaren Text und ohne `aria-label`. axe-core meldet den Verstoß *button-has-visible-text* mit dem HTML-Ausschnitt `<button class="btn-icon">...</button>` - ohne Hinweis darauf, dass die Ursache in der Datei `IconButton.tsx` liegt, wo das `aria-label`-Prop nicht weitergereicht wird.

Drittens erschwert die **Komponentenabstraktion** die Ursachenanalyse: eine wiederverwendbare Komponente mit einem Accessibility-Problem manifestiert sich im gerenderten DOM an jeder Stelle, an der sie eingesetzt wird. Accessibility-Tool-Reports melden jeden einzelnen Treffer separat, ohne den Bezug zur gemeinsamen Quellkomponente herzustellen, deren einmalige Korrektur alle Treffer beseitigen würde.¹⁰⁶

2.3.3 Playwright als Testautomatisierungswerkzeug

Um diese dynamischen Zustände automatisiert reproduzierbar testen zu können, werden Werkzeuge zur Browserautomatisierung benötigt.

¹⁰²Vgl. Tafreshipour et al. 2024, S. 110

¹⁰³Vgl. He, Huq, Malek 2025, S. 5–6; Vgl. World Wide Web Consortium (W3C) 2023, Erf.-krit. 2.4.3 & 4.1.3

¹⁰⁴Vgl. Fernandes et al. 2012, S. 31

¹⁰⁵Vgl. Fathallah, Hernández, Staab 2025

¹⁰⁶Vgl. Abou-Zahra, Brewer, Cooper 2018

Playwright ist ein von Microsoft entwickeltes Open-Source-Framework für die End-to-End-Testautomatisierung von Webanwendungen.¹⁰⁷ Es ermöglicht die automatisierte Steuerung der Browser Chromium, Firefox und WebKit über eine einheitliche API und führt Anwendungen in einem echten Browser-Kontext aus, sodass JavaScript-Ausführung, clientseitiges Rendering und dynamische DOM-Manipulationen vollständig nachgebildet werden.

Für die Barrierefreiheitsanalyse ist Playwright aus mehreren Gründen besonders geeignet. Erstens kann es durch programmierte Interaktionen - Klicks, Formularausfüllungen, Tastatureingaben, Hover-Aktionen - gezielt verschiedene Anwendungszustände auslösen.¹⁰⁸ Zweitens bietet Playwright mit `@axe-core/playwright` eine direkte Integration der axe-core-Bibliothek: nach jeder Interaktion kann unmittelbar eine vollständige axe-Prüfung des aktuellen DOM-Zustands angestoßen werden.¹⁰⁹

Verschiedene Forschungsarbeiten nutzen diese Integration bereits erfolgreich für die Barrierefreiheitsanalyse dynamischer Webanwendungen (vgl. Kapitel 3). In der vorliegenden Arbeit dient Playwright als zentrales Werkzeug der Detection-Phase: es löst definierte Interaktionssequenzen aus, erfasst die resultierenden DOM-Zustände und stößt axe-core-Prüfungen an, deren strukturierter JSON-Output als erster Input für die LLM-Analysepipeline dient.

2.4 KI-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen

Innerhalb der KI-Forschung haben sich LLMs als besonders leistungsfähige Klasse von Systemen für sprachbasierte Aufgaben etabliert.¹¹⁰ Für die vorliegende Arbeit sind sie relevant, weil sie sowohl Code verstehen als auch natürlichsprachige Handlungsempfehlungen generieren können.

2.4.1 Grundlagen: Large Language Models

LLMs sind neuronale Netzwerke, die auf extrem großen Datensätzen mit dem Ziel trainiert werden, das nächste Token einer Textsequenz vorherzusagen.¹¹¹ Durch dieses Training auf massiver Datenbasis entstehen Modelle, die komplexe Sprachstrukturen, semantische Zusammenhänge und logisches Schlussfolgern internalisieren. Für den Zugriff auf externes Wissen werden häufig *Embedding*-basierte Retrieval-Methoden verwendet (Vektorraumsuche), die in Retrieval-Augmented Generation (RAG)-Systemen zentral sind.¹¹² Die bekanntesten Vertreter sind GPT-4o (OpenAI), Claude (Anthropic) und die LLaMA-Modellfamilie (Meta).¹¹³ Entscheidend für den An-

¹⁰⁷ Vgl. Microsoft 2026

¹⁰⁸ Vgl. Microsoft 2026

¹⁰⁹ Vgl. Deque Systems 2026

¹¹⁰ Vgl. Russell, Norvig 2021, S. 22

¹¹¹ Vgl. Brynjolfsson, Li, Raymond 2023, S. 5–6

¹¹² Vgl. Lewis et al. 2021, S. 2–3

¹¹³ Vgl. Anthropic 2026a; OpenAI 2023; Touvron et al. 2023

wendungskontext dieser Arbeit sind die sogenannten Emergent Abilities dieser Modelle: Fähigkeiten wie Code-Generierung, mehrsprachige Übersetzung und kontextuelles Schlussfolgern, die ab einer bestimmten Modellgröße ohne explizites Training auftreten.¹¹⁴

Im Kontext der Barrierefreiheitsanalyse bieten LLMs gegenüber regelbasierten Tools vier wesentliche Vorteile:¹¹⁵

- Sie verfügen über ein Sprachverständnis, das die Bewertung semantischer Inhalte ermöglicht - etwa die Beurteilung, ob ein Alternativtext den Bildinhalt tatsächlich beschreibt.¹¹⁶
- Sie können Code generieren und transformieren, wodurch konkrete Handlungsempfehlungen abgeleitet werden können.¹¹⁷
- Sie können große Mengen an Text analysieren und strukturieren.¹¹⁸
- Sie verarbeiten Kontext und können durch Techniken wie Clustering die Bedeutung eines Elements im Zusammenhang der Gesamtseite beurteilen.¹¹⁹

Diesen Stärken stehen bekannte Schwächen gegenüber:

- LLMs können halluzinieren, also plausibel klingende Ausgaben erzeugen, die jedoch faktisch falsch sind.¹²⁰
- Sie sind von einem sogenannten *Knowledge-Cutoff* betroffen. Dieser bezeichnet den Stichtag, bis zu dem die Trainingsdaten eines Modells reichen, sodass dem Modell nach diesem Zeitpunkt veröffentlichte Informationen nicht bekannt sind.¹²¹ Dies gilt auch für das in dieser Arbeit eingesetzte Modell qwen2.5-coder:7b¹²², das nur den Wissensstand seines Trainingszeitraums abbildet und daher keine danach veröffentlichten WCAG-Änderungen oder Bibliotheks-Updates kennt.
- Bei gleicher Eingabe erzeugen sie nicht immer identische Ausgaben; dieses Verhalten wird als Nicht-Determinismus bezeichnet.¹²³
- Die generierten Inhalte können Verzerrungen (*Bias*) aus den Trainingsdaten übernehmen.¹²⁴

¹¹⁴Vgl. Wei, Tay et al. 2022, S. 2

¹¹⁵Vgl. Fathallah, Hernández, Staab 2025

¹¹⁶Vgl. Tafreshipour et al. 2024

¹¹⁷Vgl. Touvron et al. 2023, S. 3

¹¹⁸Vgl. Bassi et al. 2025, S. 298

¹¹⁹Vgl. Fathallah, Hernández, Staab 2025

¹²⁰Vgl. Huang, L. et al. 2025, 1:2

¹²¹Vgl. Anthropic 2026b, S. 7; Huang, L. et al. 2025, 1:9–10

¹²²Vgl. Hui et al. 2024

¹²³Vgl. OpenAI 2026; Vgl. Anthropic 2026c

¹²⁴Vgl. Bender et al. 2021, S. 614–615

Eine für die vorliegende Arbeit zentrale Frage ist, ob diese Fähigkeiten auch bei kleineren, lokal ausführbaren Modellen in ausreichendem Maße vorhanden sind. Alle in Kapitel 3 untersuchten Systeme - AccessGuru, GenA11y, ACCESS - setzen GPT-4o ein, ein Modell mit geschätzt über 200 Milliarden Parametern und Zugang zu umfangreichen Cloud-Ressourcen.¹²⁵ Lokal ausführbare Modelle der 7B-Klasse verfügen über deutlich weniger Parameter und damit potenziell über ein eingeschränkteres Sprachverständnis und schwächere Reasoning-Fähigkeiten. Ob ein solches Modell semantische Accessibility-Verstöße zuverlässig erkennen kann, ist empirisch bislang nicht belegt und wird in der vorliegenden Arbeit als explizite Forschungsfrage adressiert. Die Begründung der konkreten Modellauswahl erfolgt in Kapitel 5.4.

2.4.2 Prompt-Engineering-Strategien

Die Qualität der Ausgaben eines LLM hängt maßgeblich von der Formulierung der Eingabe - dem sogenannten Prompt - ab. Prompt Engineering bezeichnet die systematische Gestaltung dieser Eingaben, um die Ausgabequalität zu maximieren.¹²⁶ Im Folgenden werden die für diese Arbeit relevanten Strategien beschrieben, die in Kapitel 7 als Grundlage für das Prompt-Design des PoC dienen.

- **Zero-Shot Prompting** bezeichnet die direkte Anfrage ohne Beispiele oder Kontextinformationen. Das Modell nutzt ausschließlich sein vortrainiertes Wissen. Diese Strategie ist einfach umzusetzen, erzielt bei komplexen Aufgaben wie der Barrierefreiheitsanalyse jedoch häufig unzureichende Ergebnisse.¹²⁷
- **Few-Shot Prompting** ergänzt den Prompt um eine kleine Anzahl von Beispielen (Input-Output-Paaren), die dem Modell das erwartete Ausgabeformat und die inhaltlichen Erwartungen verdeutlichen. Dieser Ansatz verbessert die Konsistenz und Präzision der Ausgaben erheblich, erfordert jedoch sorgfältig ausgewählte Beispiele.¹²⁸ Als Vergleich von Zero-Shot- und Few-Shot-Prompting dient die Grafik 5.
- **Chain-of-Thought Prompting (CoT)** veranlasst das Modell, seinen Lösungsweg Schritt für Schritt zu erklären, bevor es zur finalen Antwort gelangt. Dieses explizite Reasoning reduziert nachweislich Halluzinationen bei komplexen Schlussfolgerungen und macht die Entscheidungslogik des Modells für eine Prüfung durch den Nutzer zugänglich.¹²⁹
- **ReAct-Prompting** kombiniert Reasoning (Re) und Action (Act) in einem iterativen Prozess: das Modell wechselt zwischen Denken (Reasoning) und Handeln, wie das konstante

¹²⁵Vgl. Fathallah, Hernández, Staab 2025; Vgl. He, Huq, Malek 2025, FSE101:10; Vgl. Huang, C. et al. 2024, S. 7

¹²⁶Vgl. Schulhoff et al. 2024, S. 4

¹²⁷Vgl. Brown et al. 2020, S. 4; Vgl. Njifenjou et al. 2024

¹²⁸Vgl. Brown et al. 2020, S. 5

¹²⁹Vgl. Wei, Wang, X. et al. 2022, S. 2; Fathallah, Hernández, Staab 2025

Wechseln von Tools und Daten. Dadurch werden die Ausgaben der zurückgemeldeten Ergebnisse verfeinert. Huang et al. demonstrieren, dass ReAct-Prompting bei der automatisierten Barrierefreiheitskorrektur (**ACCESS**) mit einem Severity Score Reduction von 51,3% alle anderen verglichenen Strategien übertrifft.¹³⁰

- **Role-Play Prompting** weist dem Modell eine Expertenrolle zu, die seine Ausgaben in Richtung domänenspezifischen Wissens lenkt. Ein Prompt, der das Modell als „erfahrenen Barrierefreiheitsexperten und React-Entwickler“ adressiert, verbessert nachweislich die Qualität accessibility-spezifischer Ausgaben, da das Modell seine Antworten stärker an der Perspektive eines Fachexperten ausrichtet.¹³¹
- **Metacognitive Prompting** fordert das Modell auf, die eigene Ausgabe in einem zweiten Schritt zu reflektieren und zu bewerten, bevor diese finalisiert wird.¹³² In Verbindung mit Corrective Re-Prompting - einer Technik, bei der fehlerhafte Ausgaben dem Modell zusammen mit einer Fehlerbeschreibung als Feedback zurückgespielt werden - ermöglicht dieser Ansatz eine iterative Qualitätssteigerung. Fathallah et al. belegen, dass Corrective Re-Prompting in **AccessGuru** den Violation Score Decrease von 0,72 auf 0,84 verbessert.¹³³

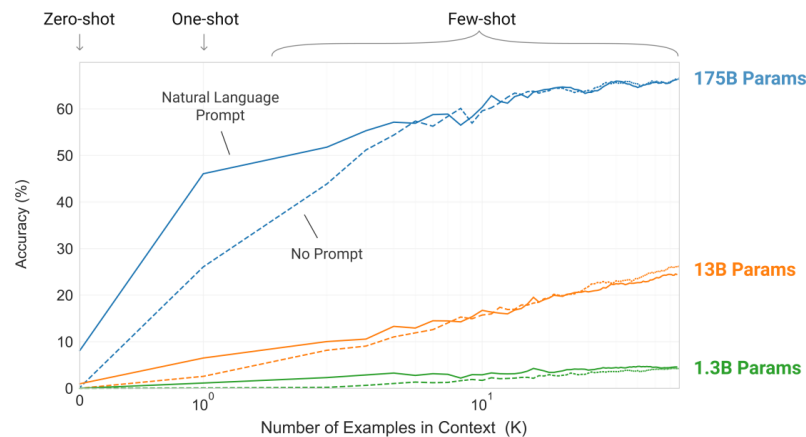


Abb. 5: Zero-Shot und Few-Shot Prompting im Vergleich¹³⁴

2.4.3 RAG und Fine-Tuning als Erweiterungsansätze

Neben Prompt-Engineering-Strategien existieren zwei weitergehende Ansätze, um die Leistungsfähigkeit von LLMs zu verbessern.

¹³⁰Vgl. Huang, C. et al. 2024, S. 5; Yao et al. 2023, S. 1

¹³¹Vgl. Fathallah, Hernández, Staab 2025; Njifenjou et al. 2024; Schulhoff et al. 2024, S. 6, 12

¹³²Vgl. Wang, Y., Zhao 2024; Schulhoff et al. 2024, S. 8–9

¹³³Vgl. Fathallah, Hernández, Staab 2025

¹³⁴Entnommen aus Brown et al. 2020

Retrieval-Augmented Generation (RAG) erweitert den Prompt eines LLM zur Laufzeit mit thematisch passenden Dokumenten, die aus einer externen Wissensbasis abgerufen werden.¹³⁵ Dabei kombiniert RAG ein Sprachmodell mit einem Retrieval-Mechanismus, der relevante Dokumente aus einem externen Korpus abrufen und als zusätzlichen Kontext in die Generierung einbindet.¹³⁶ Im Kontext der Barrierefreiheitsanalyse könnte eine RAG-Pipeline aktuelle WCAG-Erfolgskriterien, Techniken und bekannte Lösungsmuster für das Modell abrufbar machen und so sicherstellen, dass es unabhängig von seinem Trainings-Cutoff mit aktuellen Anforderungen arbeitet.

Fine-Tuning bezeichnet das domänenspezifische Training eines vortrainierten Modells auf einem kleineren, aufgabenspezifischen Datensatz.¹³⁷ Delnevo et al. argumentieren, dass ein Fine-Tuning auf einem entsprechenden Datensatz die Konsistenz von Bewertungen signifikant verbessern könnte, testen diesen Ansatz jedoch nicht empirisch.¹³⁸ Für den Kontext der öffentlichen Verwaltung wäre insbesondere ein Fine-Tuning auf BITV-spezifischen Anforderungen sowie internen Entwicklungsstandards der BITBW denkbar.

Beide Ansätze liegen außerhalb des Scopes dieser Arbeit und werden als Ausblick in Kapitel 8.1 und 8.2 aufgegriffen.

2.4.4 Datenschutz bei KI-gestützten Accessibility-Services

Der Einsatz von LLMs im Kontext öffentlicher Verwaltungen wirft spezifische Datenschutzfragen auf. Webanwendungen der öffentlichen Verwaltung verarbeiten häufig personenbezogene Daten - etwa in Formularseiten, die persönliche Angaben von Bürgerinnen und Bürgern erfassen. Werden diese Seiten für eine KI-gestützte Accessibility-Analyse an externe LLM-APIs übermittelt, ist die Wahrung der Datenschutzgarantien fraglich.

Die DSGVO schreibt vor, dass personenbezogene Daten nur auf Basis einer Rechtsgrundlage und unter Wahrung der Grundsätze der Zweckbindung und Datenminimierung verarbeitet werden dürfen.¹³⁹ Die Übermittlung von HTML-Quellcode der produktiven Anwendung, der im schlimmsten Fall personenbezogene Daten als Attributwerte enthalten kann, an externe API-Dienste wie die OpenAI API oder die Anthropic API ist ohne explizite Rechtsgrundlage und einen Auftragsverarbeitungsvertrag nach Art. 28 Datenschutz-Grundverordnung (DSGVO) für öffentliche Verwaltungen in Deutschland problematisch.¹⁴⁰

Als Lösungsansatz wird in dieser Arbeit ein Lokal-First-Prinzip verfolgt: das KI-Assistenzsystem soll ohne externe API-Aufrufe betreibbar sein, indem lokal ausführbare Open-Source-Modelle - etwa Modelle der LLaMA-Familie oder Mistral - eingesetzt werden.¹⁴¹ Damit bleiben sämtliche

¹³⁵Vgl. Lewis et al. 2021, S. 1–2

¹³⁶Vgl. Gao, Y. et al. 2023, S. 1–2

¹³⁷Vgl. Silvestre, Pietzuch 2025, S. 90

¹³⁸Vgl. Delnevo, Andruccioli, Mirri 2024

¹³⁹Vgl. Europäisches Parlament und Rat der Europäischen Union 2016

¹⁴⁰Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28

¹⁴¹Vgl. Touvron et al. 2023

Analyse- und Codedaten innerhalb der eigenen Infrastruktur. Diese Anforderung prägt sowohl die nicht-funktionalen Anforderungen (Kapitel 5) als auch die Architekturentscheidungen (Kapitel 7) dieser Arbeit.

Die in diesem Kapitel erarbeiteten theoretischen Grundlagen - das normative Fundament der WCAG, die empirisch belegten Grenzen regelbasierter Tools, die Besonderheiten von React-Anwendungen sowie die Möglichkeiten und Grenzen von LLMs und Prompt-Engineering - bilden den konzeptionellen Rahmen für die im folgenden Kapitel systematisch aufgearbeitete Forschungsliteratur und Methodik.

3 Stand der Forschung

Das folgende Kapitel gibt einen thematisch strukturierten Überblick über den Stand der Forschung zu KI-gestützter Barrierefreiheitsanalyse und arbeitet die verbleibenden Forschungslücken heraus, die den Ausgangspunkt der vorliegenden Arbeit bilden. Die relevanten Arbeiten lassen sich in drei Entwicklungslinien einordnen: (1) LLM-basierte Erkennung von Accessibility-Verstößen, (2) LLM-basierte Korrektur und Code-Generierung sowie (3) hybride Pipelines, die regelbasierte Tools mit LLM-Analyse kombinieren.

3.1 Vorgehen der Literaturrecherche

Zur Einordnung des Forschungsstands wurde eine strukturierte Literaturrecherche in wissenschaftlichen Datenbanken durchgeführt.¹⁴² Berücksichtigt wurden insbesondere ACM Digital Library, IEEE Xplore, Science Direct, arXiv und ResearchGate. Die Recherche erfolgte themenorientiert anhand von Suchbegriffen wie „web accessibility“, „LLM accessibility“, „accessibility violation detection“, „accessibility remediation“, „prompt engineering accessibility“, „Playwright accessibility“ und „axe-core“. Berücksichtigt wurden primär aktuelle Arbeiten mit Bezug zur automatisierten Erkennung oder Behebung von Barrierefreiheitsverstößen in Webanwendungen mit KI-Assistenzsystemen. Ausgeschlossen wurden Arbeiten ohne klaren Webbezug, rein konzeptionelle Beiträge ohne methodische Auswertung sowie Beiträge mit fehlender Übertragbarkeit auf den Untersuchungskontext dieser Arbeit. Die identifizierten Arbeiten wurden anschließend thematisch gruppiert, priorisiert, eingeordnet und vergleichend ausgewertet.

3.2 Von frühen KI-Ansätzen zur LLM-basierten Erkennung

Frühe Arbeiten wie Abou-Zahra et al. (2018) identifizierten zwar grundlegende Einsatzmöglichkeiten von KI für die Barrierefreiheitsanalyse, lieferten jedoch noch keine belastbaren quantitativen Leistungsmaße.¹⁴³ Mit dem Aufkommen leistungsfähiger Large Language Models verlagerte sich der Forschungsfokus auf die empirische Bewertung konkreter Modellleistungen. Delnevo et al. (2024) zeigen, dass GPT-3.5 einfache syntaktische Verstöße mit moderater Genauigkeit erkennen kann, bei semantisch anspruchsvolleren Problemen jedoch inkonsistente Bewertungen liefert.¹⁴⁴ Bassi et al. (2025) verglichen GPT-4o, GPT-3.5 und LLaMA 3.1 in einer zweistufigen Evaluation von HTML-Snippets und Webseiten. Für GPT-4o berichten sie in der ersten Prüfphase einen Anteil korrekt identifizierter Verstöße von 74%, während 15% der gemeldeten Verstöße als Halluzinationen eingestuft wurden.¹⁴⁵ Eine nachgelagerte Chain-of-Verification-artige Verifikationsstufe erhöhte die Zuverlässigkeit der Befunde deutlich, da die im ersten Schritt erkannten

¹⁴²Vgl. Webster, Watson 2002

¹⁴³Vgl. Abou-Zahra, Brewer, Cooper 2018

¹⁴⁴Vgl. Delnevo, Andruccioli, Mirri 2024

¹⁴⁵Vgl. Bassi et al. 2025, S. 302

Verstöße anschließend gezielt auf tatsächliche Fehler oder Halluzinationen überprüft wurden; ein Einsatz von RAG wird von Bassi et al. hingegen nur als mögliche zukünftige Erweiterung diskutiert.¹⁴⁶

3.3 LLM-basierte Korrektur und Prompt-Engineering

Abu Doush und Kassem (2024) zeigten, dass strukturierte Prompts die Qualität LLM-generierter barrierefreier HTML-Alternativen erheblich verbessern, jedoch bei komplexeren Interaktionsmustern weiterhin Schwächen bestehen.¹⁴⁷ Guriță und Vataavu (2025) belegten, dass accessibility-orientierte Prompts paradoxerweise zu mehr Violations führen können als neutrale Prompts; erst iteratives Prompting über fünf Runden senkte den Violation Score von 0,84 auf 0,32.¹⁴⁸

Den aktuellen Stand der Technik repräsentiert **AccessGuru** von Fathallah et al. (2025).¹⁴⁹ Das System kombiniert eine zweistufige Pipeline - regelbasierte Detection via Playwright und **axe-core**, anschließend LLM-basierte Korrektur durch GPT-4o mit Role-Play, Contextual und Metacognitive Prompting. Corrective Re-Prompting verbessert den Violation Score Decrease von 0,72 auf 0,84 bei einer semantischen Ähnlichkeit von 77% zu menschlichen Referenzlösungen. Als offenes Problem benennen die Autoren die fehlende Verknüpfung korrigierter HTML-Snippets mit dem Quellcode der geprüften Anwendung.

3.4 Hybride Pipelines und Ensemble-Testing

Huang et al. entwickelten 2024 mit **ACCESS** ein System, das Prompt Engineering gezielt zur automatisierten Barrierefreiheitskorrektur einsetzt und dabei verschiedene Strategien systematisch vergleicht.¹⁵⁰ Der Vergleich von ReAct, Chain-of-Thought und Few-Shot Prompting zeigt, dass ReAct mit einem Severity Score Reduction von 51,3% alle anderen Strategien übertrifft - ein Ergebnis, das die Bedeutung iterativer, werkzeuggestützter Reasoning-Strategien für diese Aufgabe unterstreicht.

He et al. präsentierten 2025 mit **GenA11y** das methodisch ausgefeiltste Detection-System des aktuellen Forschungsstands.¹⁵¹ **GenA11y** extrahiert nach vollständigem clientseitigem Rendering den DOM via Playwright und prüft ihn anschließend kriteriumsspezifisch mit GPT-4o. Für jedes WCAG-Erfolgskriterium wird ein eigener, optimierter Prompt eingesetzt. Eine Ablation Study belegt, dass weder DOM-Extraktion noch optimiertes Prompting allein die Gesamtleistung von Precision 94,5% und Recall 87,61% erreichen - beide Komponenten sind also notwendige

¹⁴⁶Vgl. Bassi et al. 2025, S. 301

¹⁴⁷Vgl. Doush, Kassem 2025, S. 343

¹⁴⁸Vgl. Guriță, Vataavu 2025, S. 1000

¹⁴⁹Vgl. Fathallah, Hernández, Staab 2025

¹⁵⁰Vgl. Huang, C. et al. 2024, S. 7, 9

¹⁵¹Vgl. He, Huq, Malek 2025, FSE101:20-21

Bestandteile des Systems.¹⁵² **GenA11y** erkennt acht Verstößtypen, die von keinem der verglichenen regelbasierten Tools gefunden werden.¹⁵³ Paternò et al. ergänzen dieses Bild mit **TeVal**, das semantische WCAG-Kriterien durch Role-Play Prompting mit Few-Shot-Beispielen und Confidence Score bewertet und dabei eine Effektivität von 92,4% erzielt.¹⁵⁴

Pool (2023) adressiert die Detection-Schicht mit **Testaro**, einem Ensemble aus acht Accessibility-Werkzeugen (u. a. **axe-core**, **WAVE**, **QualWeb**).¹⁵⁵ Die empirische Analyse zeigt, dass jedes Tool exklusive Violations erkennt, was die Komplementarität der Werkzeuge belegt.

3.5 Forschungslücken und Einordnung der vorliegenden Arbeit

Die Analyse der bestehenden Forschungsarbeiten zeigt drei persistente Forschungslücken, die den Ausgangspunkt der vorliegenden Arbeit definieren.

Erstens berücksichtigt keiner der untersuchten Ansätze systematisch zustandsabhängige Barrieren, die erst durch Nutzerinteraktionen in React-basierten Single-Page Applications entstehen.¹⁵⁶ Zwar setzen einzelne Systeme wie **AccessGuru** Playwright zur DOM-Extraktion ein, jedoch beschränkt sich dies auf das initiale Rendering - interaktionsbedingte Zustandswechsel werden nicht geprüft.¹⁵⁷

Zweitens verknüpft kein bekannter Ansatz Tool-Reports mit dem Quellcode der geprüften Anwendung. **AccessGuru** selbst benennt dies als offenes Problem: die generierten Korrekturen beziehen sich auf isolierte DOM-Snippets und nicht auf die JSX-Komponenten, in denen Entwicklerinnen und Entwickler tatsächlich eingreifen müssten.¹⁵⁸ Die Lücke zwischen technischer Fehlererkennung und entwicklernaher Umsetzung bleibt damit ungeschlossen.

Drittens adressiert keine der untersuchten Arbeiten die datenschutzrechtlichen Anforderungen öffentlicher Verwaltungen: alle Systeme setzen auf cloudbasierte LLM-APIs (primär GPT-4o), was für die BITBW und vergleichbare Institutionen ohne vertiefte datenschutzrechtliche Prüfung nicht nutzbar ist.¹⁵⁹

Die Konzeptmatrix in Tabelle 2 fasst die zentralen Merkmale der untersuchten Studien im Vergleich zum eigenen PoC zusammen und macht die Forschungslücken strukturell sichtbar.

¹⁵² Vgl. He, Huq, Malek 2025, FSE101:20

¹⁵³ Vgl. He, Huq, Malek 2025, FSE101:20

¹⁵⁴ Vgl. Paternò et al. 2025

¹⁵⁵ Vgl. Pool 2023

¹⁵⁶ Vgl. Tafreshipour et al. 2024, S. 110

¹⁵⁷ Vgl. Fathallah, Hernández, Staab 2025

¹⁵⁸ Vgl. Fathallah, Hernández, Staab 2025

¹⁵⁹ Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28, 44 ff.

¹⁶⁰ Vgl. Webster, Watson 2002.

Studie		Konzepte													
Autor(en) & Jahr	System	Ziel			Datenquellen			KI- / LLM-Ansatz			Evaluation			Kontext	
		WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA	Rechtl. Rahmen (BfSG etc.)
Hoch priorisierte Studien															
He et al. (2025)	GenA11y	x			x			x			x				
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x				
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x			x
Paternò et al. (2025)	TeVal	x		x	x			x	x			x			
Tafreshipour et al. (2024)	Ma11y	x			x	x					x				
Fernández-Navarro & Chicano (2025)	LLM Remediation	x	x	x	x		x	x			x			x	
Martinez Campo (2026)	ACE (angestrebt)	x	x	x	x	x	x	x	x	x		x		x	x

Tab. 2: Konzeptmatrix nach Webster Watson¹⁶⁰: hoch priorisierte Studien im Vergleich zum eigenen Proof of Concept (PoC). x = Konzept vorhanden/adressiert; leer = nicht adressiert. Für den eigenen PoC bezeichnen die Markierungen angestrebte Designziele, deren Umsetzung und Wirksamkeit in Kapitel 8 evaluiert wird. Die vollständige Matrix aller analysierten Studien findet sich in Anhang 2.

4 Methodik

Das folgende Kapitel legt das methodische Vorgehen der Arbeit dar. Zunächst wird die gewählte Forschungsmethode begründet (Abschnitt 4.1), anschließend der Untersuchungsgegenstand abgegrenzt (Abschnitt 4.2) und schließlich das Evaluationsdesign mit den zugehörigen Bewertungskriterien definiert (Abschnitt 4.3).

4.1 Wissenschaftliche Methode

Die vorliegende Arbeit folgt dem Forschungsparadigma des Design Science Research (DSR), das von Hevner et al. für die Wirtschaftsinformatik systematisiert wurde.¹⁶¹ DSR eignet sich für diese Arbeit aus zwei Gründen: Erstens steht die Konstruktion eines konkreten technischen Artefakts - des Proof of Concept zur KI-gestützten Barrierefreiheitsanalyse - im Zentrum der Forschungsfrage, nicht die Überprüfung einer statistischen Hypothese. Zweitens fordert DSR explizit die Evaluation des Artefakts in einem realen Anwendungskontext, was dem praxisorientierten Charakter der Arbeit entspricht.¹⁶² Alternative Forschungsansätze wie Action Research oder kontrollierte Experimente wurden verworfen: Action Research setzt eine iterative Zusammenarbeit mit Praktikern über mehrere Zyklen voraus, die im Rahmen einer Bachelorarbeit nicht realisierbar ist.¹⁶³ Ein kontrolliertes Experiment würde eine statistisch belastbare Stichprobe vergleichbarer Webanwendungen erfordern, die für den spezifischen Kontext öffentlicher Verwaltungen nicht verfügbar ist.

Als prozessualer Rahmen dient das sechsstufige DSRM-Prozessmodell nach Peffers et al. Tabelle 3 ordnet die Phasen den jeweiligen Kapiteln dieser Arbeit zu.¹⁶⁴

DSRM-Phase	Kapitel
Problem-Identifikation	Kapitel 1-3
Zieldefinition	Kapitel 1.3 & 3.5
Design & Entwicklung	Kapitel 5-??
Demonstration	Kapitel 7
Evaluation	Kapitel 8
Kommunikation	Kapitel 9

Tab. 3: Zuordnung der DSRM-Phasen zu den Kapiteln der Arbeit.

Ergänzt wird dieser Rahmen durch einen Fallstudienansatz nach Yin.¹⁶⁵ Das von der BITBW entwickelte Fachverfahren BW-Empfangsclient dient als Einzelfallstudie, an der das entwickelte

¹⁶¹Vgl. Hevner et al. 2004

¹⁶²Vgl. Hevner et al. 2004, S. 80

¹⁶³Vgl. Baskerville, R. L. 1999, S. 1–2; Vgl. Baskerville, R., Myers 2004, S. 239–335

¹⁶⁴Vgl. Peffers et al. 2007

¹⁶⁵Vgl. Yin 2018

System demonstriert und evaluiert wird. Der Fallstudienansatz ist geeignet, weil die Forschungsfrage auf ein zeitgenössisches Phänomen (KI-gestützte Accessibility-Analyse) in einem spezifischen Praxiskontext (öffentliche Verwaltung, React-Anwendung) abzielt, bei dem die Grenzen zwischen technischem System und organisatorischem Kontext nicht trennscharf sind.

Die Wahl eines PoC als Artefakttyp ist durch die explorative Natur der Forschungsfrage begründet: das Ziel ist die konzeptionelle Demonstration der technischen Machbarkeit, nicht die statistisch belastbare Generalisierung auf eine Grundgesamtheit. Dieses Vorgehen entspricht dem in der Wirtschaftsinformatik etablierten Ansatz für frühe Forschungsphasen, in denen zunächst die Funktionsfähigkeit eines Artefakts nachzuweisen ist, bevor großangelegte Wirkungsstudien durchgeführt werden können.¹⁶⁶

4.2 Untersuchungsgegenstand und Scope-Abgrenzung

Als Untersuchungsgegenstand dient der BWEC, eine öffentlich zugängliche React-basierte Webanwendung der BITBW.¹⁶⁷ Die Anwendung eignet sich als Fallstudie, da sie drei für die Forschungsfrage zentrale Eigenschaften vereint: gesetzlich relevante Anforderungen an digitale Barrierefreiheit, einen datenschutzsensiblen Einsatzkontext in der öffentlichen Verwaltung sowie typische Interaktionsmuster moderner Single-Page Applications.

Der Untersuchungsscope ist wie folgt abgegrenzt:

- **Eingeschlossen:** Öffentlich zugängliche Bereiche der Anwendung ohne Authentifizierung. Betrachtet werden Barrierefreiheitsverstöße bezüglich WCAG 2.2 auf Konformitätsstufe A und AA, soweit sie durch regelbasierte Prüfung, DOM-Analyse und Interaktionsausführung im Frontend beobachtbar sind.¹⁶⁸
- **Ausgeschlossen:** Backend-Logik, Datenbankstrukturen, organisatorische Betriebsprozesse sowie eine vollständige rechtliche Bewertung im Sinne einer formalen BITV-Prüfung.¹⁶⁹

Die Arbeit erhebt keinen Anspruch auf vollständige Abdeckung aller WCAG-Erfolgskriterien oder auf Generalisierbarkeit auf beliebige Frameworks und Anwendungstypen. Diese Einschränkungen werden in Kapitel 8 als Limitationen diskutiert.

4.3 Evaluationsdesign und Kriterien

Ziel der Evaluation ist es, die technische Machbarkeit und den praktischen Nutzen des entwickelten PoC im Anwendungskontext des BWEC zu untersuchen.

¹⁶⁶Vgl. Hevner et al. 2004

¹⁶⁷Vgl. Land Baden-Württemberg 2025

¹⁶⁸Vgl. World Wide Web Consortium (W3C) 2023

¹⁶⁹Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

4.3.1 Evaluationslogik

Die Evaluation folgt einem fünfstufigen Vorgehen:

1. **Seitenauswahl:** Relevante öffentlich zugängliche Seiten und Interaktionszustände des BWEC werden für die Untersuchung ausgewählt.
2. **Manuelles Referenzaudit:** Der Autor führt ein manuelles Audit der ausgewählten Seiten durch, um vorhandene Barrierefreiheitsverstöße zu identifizieren und nach Verstoßtyp (syntaktisch, semantisch, interaktionsbezogen) zu kategorisieren. Als Prüfgrundlage dienen die WCAG 2.2-Erfolgskriterien der Stufen A und AA.¹⁷⁰
3. **Automatisierte Analyse:** Dieselben Seiten werden mit dem entwickelten PoC analysiert.
4. **Vergleich:** Die Ergebnisse des Systems werden dem manuellen Audit gegenübergestellt. Dabei wird erfasst, welche Verstöße korrekt erkannt, welche übersehen und welche fälschlich gemeldet werden.
5. **Wirksamkeitsprüfung:** Eine Stichprobe der generierten Handlungsempfehlungen wird manuell in repräsentativen Codeausschnitten umgesetzt und anschließend erneut mit axe-core geprüft, um den Violation Score Decrease zu messen.¹⁷¹

4.3.2 Bewertungskriterien

Die Evaluation erfolgt qualitativ anhand von vier Kriterien, die auf Basis der Forschungsfragen und der Fachliteratur entwickelt wurden.

Kriterium 1 - Korrektheit der Erkennung: Die vom System identifizierten Verstöße werden mit dem manuellen Referenzaudit verglichen. Dabei wird differenziert, welche Verstoßtypen erkannt werden und wo das System versagt. Als Orientierungswerte dienen die Ergebnisse vergleichbarer Systeme: **GenA11y** erreicht eine Precision von 94,5% und einen Recall von 87,61%.¹⁷²

Kriterium 2 - Qualität der Handlungsempfehlungen: Die generierten Empfehlungen werden hinsichtlich Verständlichkeit, Korrektheit und direkter Umsetzbarkeit beurteilt. Quantitativ wird der Violation Score Decrease durch axe-core-Nachprüfung nach Umsetzung einer Stichprobe gemessen. **AccessGuru** erreicht als Referenzwert einen Violation Score Decrease von 84%.¹⁷³

¹⁷⁰Vgl. World Wide Web Consortium (W3C) 2023

¹⁷¹Vgl. Deque Systems 2026; Vgl. Pool 2023; Vgl. Kodithuwakku, Wickramaarachchi 2025

¹⁷²Vgl. He, Huq, Malek 2025, FSE101:20

¹⁷³Vgl. Fathallah, Hernández, Staab 2025

Kriterium 3 - Mehrwert der Kontextualisierung: Um den Einfluss der Quellcode- und Playwright-Kontextualisierung isoliert bewerten zu können, wird ein Ablationsvergleich durchgeführt: dieselben Testfälle werden in zwei Konfigurationen analysiert - einmal mit dem vollständigen Kontext (axe-Report, Playwright-Interaktionsdaten und Quellcode) und einmal ausschließlich mit dem axe-Report und dem gerenderten DOM-Snippet. Die resultierenden Handlungsempfehlungen beider Durchläufe werden anhand der Kriterien Quellcodebezug, Umsetzbarkeit und semantische Tiefe qualitativ verglichen. Dies adressiert direkt Teilforschungsfrage TF2.¹⁷⁴

Kriterium 4 - Technische Praktikabilität: Laufzeit, Stabilität und Qualität des JSON-Outputs werden unter realen Bedingungen dokumentiert.

4.3.3 Limitationen des Evaluationsdesigns

Zwei methodische Einschränkungen sind vorab zu benennen. Erstens wird die Bewertung durch den Autor selbst durchgeführt und nicht durch unabhängige Gutachter validiert. Zweitens sind die genannten Referenzwerte von **AccessGuru**¹⁷⁵ und **GenA11y**¹⁷⁶ nur eingeschränkt vergleichbar, da sie auf anderen Datensätzen und unter anderen Rahmenbedingungen ermittelt wurden. Sie dienen daher als Orientierung, nicht als direkter Benchmark. Beide Limitationen werden in Kapitel 8 vertieft diskutiert.

¹⁷⁴Siehe Kapitel 1.3

¹⁷⁵Vgl. Fathallah, Hernández, Staab 2025

¹⁷⁶Vgl. He, Huq, Malek 2025

5 Analyse und Anforderungen

In diesem Kapitel werden die Grundlagen für das in Kapitel ?? beschriebene Systemdesign erarbeitet. Abschnitt 5.1 analysiert den Problemkontext und leitet daraus den konkreten Handlungsbedarf ab. Abschnitte 5.2 und 5.3 spezifizieren funktionale bzw. nicht-funktionale Anforderungen an das zu entwickelnde System. Abschnitt 5.4 begründet die Auswahl des eingesetzten Sprachmodells.

5.1 Problemkontext und Handlungsbedarf

Die BITBW entwickelt und betreibt als zentraler IT-Dienstleisterin der Landesverwaltung Baden-Württemberg eine Vielzahl webbasierter Anwendungen.¹⁷⁷ Alle diese Anwendungen unterliegen den gesetzlichen Anforderungen der BITV 2.0, die WCAG 2.2 auf Konformitätsstufe AA als verbindlichen Maßstab definiert.¹⁷⁸ In der Praxis erfolgt die Barrierefreiheitsprüfung heute überwiegend manuell oder durch den Einsatz einzelner regelbasierter Tools - ein Vorgehen, das in zweifacher Hinsicht problematisch ist: manuelle Audits sind personalintensiv und skalieren unzureichend mit der Anzahl zu betreuender Anwendungen; regelbasierte Tools erfassen, wie in Kapitel 2.2.1 dargelegt, nur einen Bruchteil tatsächlich vorhandener Barrieren.¹⁷⁹

Im Sinne des Design-Science-Research-Paradigmas ist der Ausgangspunkt die Identifikation eines praxisrelevanten Problems, für das bestehende Lösungen unzureichend sind.¹⁸⁰ Aus der Analyse der Forschungsliteratur (Kapitel 3) und den Rahmenbedingungen der BITBW lassen sich drei zentrale Problemfelder ableiten.¹⁸¹

Zum einen ist in der betrachteten Literatur kein Ansatz erkennbar, der nicht nur den anfänglichen DOM-Zustand prüft, sondern auch Barrieren erkennt, die erst durch typische Nutzerinteraktionen in React-Anwendungen sichtbar werden.¹⁸² Zum anderen fehlt bislang eine Lösung, die die Ergebnisse einer allgemeinen DOM-Analyse so verständlich und praxisnah aufbereitet, dass Entwickler gezielt für einzelne Quellcodedateien und Komponenten konkrete Verbesserungen ableiten können. Außerdem setzen alle bekannten Forschungsprototypen auf cloudbasierte LLM-APIs - ein Ansatz, der für die BITBW aus Datenschutzgründen nicht ohne Weiteres in Frage kommt.^{183 184}

¹⁷⁷ Vgl. BITBW 2021

¹⁷⁸ Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

¹⁷⁹ Vgl. Tafreshipour et al. 2024, S. 8-9

¹⁸⁰ Vgl. Hevner et al. 2004, S. 79

¹⁸¹ Vgl. Peffers et al. 2007, S. 52-55

¹⁸² Vgl. Fathallah, Hernández, Staab 2025

¹⁸³ Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28, 44 ff.

¹⁸⁴ Siehe Kapitel 2.4.4

Aus diesen drei Dimensionen leitet sich das Ziel der vorliegenden Arbeit ab: die Entwicklung eines lokal ausführbaren, LLM-gestützten Assistenzsystems, das axe-core-Violation-Reports, Playwright-Interaktionsdaten und statische Quellcodeanalyse kombiniert, um entwicklernahe Handlungsempfehlungen für React-basierte Webanwendungen zu generieren.¹⁸⁵

5.2 Funktionale Anforderungen (FA)

Funktionale Anforderungen (FA) beschreiben, was ein System leisten soll - d. h. welche Funktionen und Dienste es seinen Nutzerinnen und Nutzern bereitstellt.¹⁸⁶ Die folgenden Anforderungen wurden aus der Problemanalyse (Abschnitt 5.1), den Forschungslücken (Abschnitt 3.5) sowie einer Analyse vergleichbarer Systeme aus der Literatur abgeleitet.

FA-01 Regelbasierte Accessibility-Erkennung

Das System muss den BVEC automatisiert auf Barrierefreiheitsverstöße nach WCAG 2.2 Level A und AA prüfen.¹⁸⁷ Die Prüfung basiert auf axe-core als Detection-Backend. Dessen Regelkatalog deckt einen wesentlichen Teil der syntaktisch und algorithmisch prüfbaren Verstöße gegen WCAG 2.0, 2.1 und 2.2 ab¹⁸⁸, ersetzt jedoch keine vollständige manuelle Barrierefreiheitsprüfung.¹⁸⁹ Der Output je Verstoß umfasst mindestens: Regel-ID (Zur Identifikation), Schweregrad (critical/serious/moderate/minor) sowie ein HTML-Ausschnitt des betroffenen Elements.

FA-02 Dynamische Zustandserfassung via Playwright

Das System muss über Playwright definierte Interaktionssequenzen ausführen. Darunter fallen Seitenaufbau, Formularbefüllung, Absende-Aktionen und Fokus-Navigation, die für jeden resultierenden DOM-Zustand eine axe-core-Prüfung anstoßen.¹⁹⁰ Damit werden Barrieren erfasst, die im initialen Seitenzustand nicht vorhanden sind.¹⁹¹

FA-03 Statische Quellcodeanalyse

Das System muss den Quellcode der geprüften React-Anwendung automatisiert nach Mustern durchsuchen, die auf Barrierefreiheitsprobleme hindeuten: fehlende `aria`-Attribute, `onClick`-Handler auf Nicht-interaktiven Elementen, fehlende `alt`-Attribute, Verwendung veralteter HTML-Elemente sowie fehlende Formular-Assoziationen.¹⁹²

FA-04 LLM-gestützte Analyse und Handlungsempfehlung

¹⁸⁵Siehe Kapitel 1.3

¹⁸⁶Vgl. Sommerville 2016, S. 6-9

¹⁸⁷Vgl. Land Baden-Württemberg 2025; World Wide Web Consortium (W3C) 2023

¹⁸⁸Vgl. World Wide Web Consortium (W3C) 2023

¹⁸⁹Vgl. Deque Systems 2026

¹⁹⁰Vgl. Microsoft 2026

¹⁹¹Siehe Kapitel 2.3.2

¹⁹²Siehe Kapitel 2.1

Das System muss die Ergebnisse aus FA-01, FA-02 und FA-03 zu einem strukturierten Kontext zusammenführen und an ein lokal ausgeführtes Sprachmodell übergeben. Das Sprachmodell erhält einen kombinierten Prompt¹⁹³ und generiert daraus eine Handlungsempfehlung im Format Must-have/Nice-to-have je Verstoß und mit Angabe der betroffenen Datei.

FA-05 Strukturierter JSON-Output

Das System muss seine Ergebnisse in einem maschinenlesbaren JSON-Format ausgeben.¹⁹⁴ Der Report enthält je Befund: Verstoß-ID, Kategorie (syntaktisch/semantisch/layout)¹⁹⁵¹⁹⁶, Schweregrad, betroffene Datei(en), Handlungsempfehlung (Must-have/Nice-to-have), WCAG-Erfolgskriterium und - falls vorhanden - den relevanten Quellcode-Ausschnitt. Das strukturierte Format erleichtert die Weiterverarbeitung und Integration der Ergebnisse in bestehende Entwicklungsprozesse.

5.3 Nicht-funktionale Anforderungen (NFA)

Nicht-funktionale Anforderungen (NFA) beschreiben Qualitätseigenschaften des Systems, die bestimmen, wie gut es seine funktionalen Aufgaben erfüllt. Sie haben für das vorliegende System besonderes Gewicht, da der Betriebskontext der BITBW (öffentliche Verwaltung, lokale Infrastruktur) strenge Rahmenbedingungen setzt.

NFA-01 Datenschutz und Lokal-First

Das System muss vollständig ohne externe API-Aufrufe betreibbar sein. Weder Quellcode noch DOM-Inhalte noch Violation-Reports dürfen das interne Netzwerk verlassen. Diese Anforderung ergibt sich aus den Vorgaben der DSGVO zur Auftragsverarbeitung und zur Sicherheit der Verarbeitung, da die analysierten Anwendungen potenziell personenbezogene Daten, etwa in Formularinhalten, enthalten können.¹⁹⁷

NFA-02 Laufzeit und Praktikabilität

Der vollständige Analyselauf - von der Playwright-Initialisierung bis zum fertigen JSON-Report - muss auf einem Entwicklungsrechner in einer für den Entwicklungsalltag akzeptablen Zeit abschließen. Als Richtwert gilt eine Gesamtlaufzeit von unter zehn Minuten für eine einzelne Seite mit bis zu 20 axe-Verstößen. Diese Anforderung begrenzt die Komplexität der eingesetzten Prompt-Kette und schließt mehrstufige Reasoning-Schleifen (wie iteratives Corrective Re-Prompting)¹⁹⁸ aus dem Scope des PoC aus.

NFA-03 Wartbarkeit und Erweiterbarkeit

¹⁹³Siehe Kapitel ??

¹⁹⁴Vgl. Paternò et al. 2025; Pool 2023

¹⁹⁵Vgl. Fathallah, Hernández, Staab 2025

¹⁹⁶Siehe Kapitel 2.2.3

¹⁹⁷Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28 und Art. 32

¹⁹⁸Siehe Kapitel 2.4.2

Die Systemarchitektur muss eine klare modulare Trennung der Komponenten (Detection, Context-Building, Synthesis) vorsehen, sodass einzelne Komponenten - etwa die Playwright-Skripte oder der Prompt - unabhängig angepasst oder ausgetauscht werden können.

NFA-04 Reproduzierbarkeit

Das System muss deterministisch genug sein, dass zwei aufeinanderfolgende Analyseläufe auf derselben Anwendungsversion im Wesentlichen dieselben Befunde produzieren. Da LLMs inhärent probabilistisch sind, wird Reproduzierbarkeit durch eine feste Temperatur-Einstellung (Temperature = 0.1) für das Sprachmodell und durch strukturierte Ausgabeformate (JSON-Schema im Prompt) sichergestellt.¹⁹⁹

5.4 Modellauswahl: Ollama und qwen2.5-coder:7b

Die Wahl des Sprachmodells ist für das vorliegende System von besonderer Relevanz, da NFA-01 die Nutzung cloudbasierter Modelle ausschließt und gleichzeitig FA-04 ein Modell erfordert, das sowohl Code verstehen als auch natürlichsprachige Handlungsempfehlungen generieren kann.

5.4.1 Ollama als Laufzeitumgebung

Ollama ist ein Open-Source-Werkzeug, das die lokale Ausführung von LLMs über eine einheitliche REST-API ermöglicht.²⁰⁰ Die Modellverwaltung (Download, Quantisierung, Speicherverwaltung) wird durch eine einfach zu bedienende Command Line Interface (CLI) abstrahiert: Modelle lassen sich mit `ollama pull <modell>` herunterladen und mit `ollama run <modell>` starten.²⁰¹ Die REST-API ist kompatibel mit der OpenAI-API-Spezifikation, wodurch sich viele bestehende Bibliotheken und Anwendungen - insbesondere in Node.js - ohne größeren Anpassungsaufwand integrieren lassen. Für den vorliegenden PoC ist Ollama die einzige zusätzliche Infrastruktur, die neben einer Standard-Node.js-Installation benötigt wird.

5.4.2 Begründung der Modellwahl

Für den Einsatz im vorliegenden System wurden lokal ausführbare Modelle der 7B-Parameter-Klasse verglichen, da diese auf Entwicklungsrechnern ohne dedizierte Grafikkarte (GPU) mit akzeptabler Latenz betrieben werden können. Alternativ wurden LLaMA 3.1-8B und Mistral-7B als Kandidaten in Betracht gezogen.

Qwen2.5-Coder ist eine von Alibaba Cloud entwickelte, speziell auf Code-Aufgaben ausgerichtete Modellfamilie.²⁰² Das Modell wurde auf über 5,5 Billionen Tokens trainiert, darunter ein hoher

¹⁹⁹ Vgl. OpenAI 2026, Temperature; Vgl. Anthropic 2026c, Temperature

²⁰⁰ Vgl. Touvron et al. 2023, S. 1

²⁰¹ Vgl. Ollama 2026

²⁰² Vgl. Hui et al. 2024, S. 3

Anteil an Code-Daten aus öffentlichen Repositories, Stack Overflow und technischen Dokumentationen.²⁰³ In standardisierten Code-Benchmarks - darunter HumanEval, MBPP und EvoEval - übertrifft Qwen2.5-Coder:7B vergleichbare Modelle der 7B-Klasse inklusive CodeLlama-7B, DeepSeek-Coder-6.7B, LLaMA-3.1-8B und Mistral-7B konsistent.²⁰⁴

Für den vorliegenden Anwendungsfall sind drei Eigenschaften entscheidend: erstens versteht qwen2.5-coder:7b JSX und TypeScript nativ, was für die Analyse von React-Quellcode ohne Vorverarbeitung relevant ist. Zweitens erzeugt das Modell strukturierte JSON-Ausgaben mit hoher Zuverlässigkeit, was NFA-04 (Reproduzierbarkeit) begünstigt. Drittens beträgt das Kontextfenster standardmäßig 32.768 Tokens, was ausreicht, um mehrere Violation-Reports, relevante Quellcode-Ausschnitte und den System-Prompt in einer einzigen Anfrage zu übermitteln.²⁰⁵

Es ist jedoch zu betonen, dass diese Benchmarks allgemeine Code-Generierung messen und keine Aussage über die Fähigkeit zur semantischen Accessibility-Analyse erlauben. Ob ein 7B-Modell die in Kapitel 2.4 beschriebenen Vorteile von LLMs - insbesondere Sprachverständnis und kontextuelles Schlussfolgern - in ausreichendem Maße besitzt, ist eine offene Frage, die durch die Evaluation in Kapitel 8 empirisch adressiert wird.

Die Wahl eines 7B-Modells anstelle eines leistungsfähigeren, aber ressourcenintensiveren Modells (z. B. LLaMA-3.1-70B) ist bewusst: im Rahmen eines PoC, der auf handelsüblicher Hardware betreibbar sein muss, ist die Praktikabilität ein entscheidendes Auswahlkriterium. Die datenschutzrechtliche Anforderung (NFA-01) schließt zudem cloudbasierte Alternativen vollständig aus.

5.4.3 Zusammenfassung der Anforderungen

Tabelle 4 fasst die spezifizierten Anforderungen zusammen und gibt jeweils den Ursprung und die Priorität an.

Die Anforderungen FA-01 bis FA-05 und NFA-01 bis NFA-04 bilden die verbindliche Grundlage für das Systemdesign in Kapitel ???. NFA-01 (Lokal-First) hat dabei Vorrang vor allen anderen Anforderungen und schränkt den Lösungsraum strukturell ein: alle Designentscheidungen müssen mit dem Betrieb auf lokaler Infrastruktur vereinbar sein.

²⁰³Vgl. Hui et al. 2024, S. 4

²⁰⁴Vgl. Hui et al. 2024, S. 19

²⁰⁵Vgl. Hui et al. 2024, S. 13

ID	Beschreibung	Ursprung	Priorität
Funktionale Anforderungen			
FA-01	Regelbasierte Accessibility-Erkennung via axe-core	Forschungsliteratur (Kap. 3)	Muss
FA-02	Dynamische Zustandserfassung via Playwright	Forschungslücke 1 (Kap. 3.5)	Muss
FA-03	Statische Quellcodeanalyse (grep-basiert)	Forschungslücke 2 (Kap. 3.5)	Muss
FA-04	LLM-gestützte Analyse und Handlungsempfehlung	Forschungsliteratur (Kap. 3)	Muss
FA-05	Strukturierter JSON-Output	Praxisanforderung BITBW	Muss
Nicht-funktionale Anforderungen			
NFA-01	Datenschutz und Lokal-First (kein externer API-Aufruf)	Institutionell (DSGVO, BITBW)	Muss
NFA-02	Laufzeit < 10 min pro Seite (ohne GPU)	Praxisanforderung BITBW	Soll
NFA-03	Wartbarkeit und Erweiterbarkeit (modulare Architektur)	Software-Engineering Best Practice	Soll
NFA-04	Reproduzierbarkeit (Temperature = 0.1, JSON-Schema)	Forschungsmethodik	Soll

Tab. 4: Anforderungsübersicht: funktionale und nicht-funktionale Anforderungen mit Ursprung und Priorität.

6 Konzeption des Assistenzsystems

Das vorliegende Kapitel beschreibt die konzeptionelle Grundlage des entwickelten Assistenzsystems. Ausgehend von den in Kapitel 4 ermittelten Anforderungen werden zunächst die Zielarchitektur und die Datenpipeline erläutert, bevor das Modell der Kontextualisierung und Priorisierung sowie das gewählte Ausgabeformat vorgestellt werden. Jede Entwurfsentscheidung wird dabei mit Bezug auf die identifizierten funktionalen und nicht-funktionalen Anforderungen begründet.

6.1 Zielarchitektur und Datenpipeline

Die Zielarchitektur des Systems folgt dem Prinzip einer sequenziellen Datenpipeline, die drei voneinander unabhängige Analysequellen zusammenführt und deren Ergebnisse in einem einheitlichen Schema normalisiert, bevor sie an ein lokales Sprachmodell übergeben werden. Abbildung ?? illustriert den Gesamtablauf.

Die Pipeline gliedert sich in vier Phasen: die parallele Datenerhebung durch `axe-core`, `Playwright` und `grep`, die Normalisierung der Rohdaten in das Unified Finding Schema (UFS), die Prompt-Konstruktion mit Token-Budget-Steuerung sowie die Inferenz durch das lokale Sprachmodell und die anschließende Ausgabe.

6.1.1 Analysequellen

Das System nutzt drei komplementäre Werkzeuge zur Datenerhebung, die jeweils unterschiedliche Aspekte der Barrierefreiheit abdecken und sich gegenseitig ergänzen.

axe-core wird über einen gesteuerten Chromium-Browser ausgeführt und analysiert den vollständig gerenderten DOM der Zielanwendung auf Verstöße gegen WCAG-Erfolgskriterien. Da Rich-Client-Webanwendungen ihre Inhalte erst zur Laufzeit per JavaScript erzeugen, ist ein echter Browser-Launch unerlässlich – statische HTML-Analysen würden den tatsächlichen Zustand der Anwendung nicht abbilden.²⁰⁶

Playwright führt sieben generische Interaktionschecks aus, die unabhängig von der konkreten Anwendungslogik auf jeder Web-App lauffähig sind. Geprüft werden unter anderem die Erreichbarkeit aller interaktiven Elemente per Tastatur, die Sichtbarkeit des Fokusindikators sowie das Vorhandensein eines Skip-Links. Diese verhaltensbasierten Checks decken Verletzungen auf, die im statischen DOM nicht sichtbar sind, etwa fehlende Fokus-Traps in modalen Dialogen.²⁰⁷

²⁰⁶Vgl. Fathallah, Hernández, Staab 2025

²⁰⁷Vgl. Huang, C. et al. 2024

grep durchsucht den React-Quellcode der Anwendung nach bekannten Anti-Patterns und reichert gleichzeitig axe-core-Findings um ihren Entstehungskontext im Quellcode an. Die Entscheidung für reguläre Ausdrücke anstelle eines vollständigen Abstract Syntax Tree (AST)-Parsers ergibt sich aus zwei Überlegungen: Erstens sind die sechs implementierten Pattern-Checks syntaktischer Natur – sie suchen nach textuell erkennbaren Mustern im JSX-Markup, nicht nach semantischen Datenflüssen, für die ein AST-Parser erforderlich wäre. Zweitens gewährleistet der regexp-basierte Ansatz Sprachunabhängigkeit über `.tsx`, `.jsx`, `.ts` und `.js`-Dateien hinweg, ohne sprachspezifische Parser-Abhängigkeiten einzuführen.²⁰⁸ Die Anreicherungsstrategie folgt dabei einem selektiven Ansatz: Durchsucht werden ausschließlich jene Quelldateien, auf die axe-core oder Playwright hinweisen, um das verfügbare Token-Budget nicht durch irrelevante Codefragmente zu belasten (vgl. Abschnitt 6.2).

6.1.2 Unified Finding Schema

Ein zentrales architektonisches Element ist das *Unified Finding Schema* (UFS), das alle Findings unabhängig von ihrer Herkunft in eine einheitliche TypeScript-Schnittstelle überführt. Ohne eine solche Normalisierungsschicht würde der Prompt-Builder drei strukturell heterogene Datenformate erhalten: axe-core liefert Objekte mit **impact**-Stufen und DOM-Selektorketten, Playwright gibt boolesche **passed**-Felder mit Check-IDs zurück, und grep produziert Datei-Zeilen-Tupel mit Textfragmenten. Diese Heterogenität hätte drei negative Konsequenzen.

Erstens würde die Token-Effizienz sinken, da rohe axe-core-Ausgaben vollständige DOM-Snapshots und ausführliche Hilfetexte enthalten, die für die LLM-Inferenz nicht relevant sind. Zweitens wäre eine quellenübergreifende Deduplizierung nicht möglich, da kein gemeinsames Identifikationsfeld existierte. Drittens könnte die Severity-basierte Priorisierung im Token-Budget-Algorithmus nicht angewendet werden, da **impact** (axe), **severity** (intern) und Dateiname (grep) inkommensurable Größen darstellen.²⁰⁹

Das UFS definiert folgende Kernfelder:

- **source**: Herkunftsquelle (`"axe"`, `"playwright"`, `"grep"`)
- **ruleId**: Eindeutiger Regelbezeichner, normalisiert über alle Quellen
- **severity**: Einheitliche Schweregradbewertung (`critical`, `serious`, `moderate`, `minor`)
- **wcagCriteria** und **wcagLevel**: Zugeordnetes WCAG-Erfolgskriterium und Konformitätsstufe
- **selector**: CSS-Selektor des betroffenen DOM-Elements
- **componentPath**: Dateipfad der React-Komponente (sofern durch grep ermittelt)

²⁰⁸bassi_2025

²⁰⁹schulhoff_prompt_2024

- `codeSnippet` und `lineRange`: Relevanter Quellcodeausschnitt mit Zeilenangabe
- `category`: Klassifikation nach "syntaktisch", "semantisch" oder "layout" in Anlehnung an die Taxonomie von²¹⁰

Das Feld `componentPath` ist dabei das entscheidende Bindeglied zwischen DOM-Analyse und Quellcode: Erst durch die Verknüpfung eines `axe-Findings` mit dem konkreten Dateipfad und der Zeilennummer wird aus einer generischen Meldung („*Element hat keinen alternativen Text*“) eine entwicklernahe Handlungsanweisung („*In LoginForm.tsx, Zeile 46: Füge alt-Attribut hinzu*“).

6.2 Modell der Kontextualisierung und Priorisierung

6.2.1 Single-Prompt-Strategie

Eine grundlegende Entwurfsentscheidung betrifft die Interaktionsstrategie mit dem lokalen Sprachmodell. Konzeptionell wäre ein mehrstufiger Ansatz denkbar, bei dem zunächst die `axe-Findings`, dann die Playwright-Ergebnisse und abschließend der Quellcodekontext in separaten Aufrufen verarbeitet werden, wie es etwa Fathallah, Hernández, Staab mit dem AccessGuru-System über eine Cloud-API realisieren.²¹¹

Für den vorliegenden Anwendungsfall wird hingegen eine Single-Prompt-Strategie gewählt, die alle drei Datenquellen in einem einzigen Modellaufruf zusammenführt. Diese Entscheidung ergibt sich aus drei Überlegungen.

Latenzbudget: Die Laufzeitmessungen zeigen, dass ein einzelner LLM-Aufruf auf einem lokalen CPU-basierten System zwischen 491 und 3.238 Sekunden benötigt, abhängig von der Modellgröße. Eine Multi-Step-Kette mit vier sequenziellen Aufrufen würde die Gesamtlaufzeit auf ein Vielfaches erhöhen und damit NFA-02 (maximale Analysedauer von 20 Minuten) noch deutlicher verletzen, als es beim größten getesteten Modell ohnehin der Fall ist.

Cross-Source-Korrelation: Ein einzelner Prompt ermöglicht dem Sprachmodell, Zusammenhänge zwischen den Quellen zu erkennen und zu einem kohärenten Fix zusammenzufassen. Meldet `axe-core` ein fehlendes `aria-label` und liefert `grep` gleichzeitig den zugehörigen Quellcodeausschnitt, kann das Modell beides in einem einzigen, konkreten Vorschlag verbinden. Bei getrennten Aufrufen ginge dieser Kontext verloren.²¹²

Auditierbarkeit: Ein deterministischer, reproduzierbarer Prompt ist für eine wissenschaftliche Evaluation wichtiger als hypothetische Qualitätsgewinne durch mehrstufige Verarbeitung. Ein Prompt entspricht einem nachvollziehbaren Entscheidungspfad; Fehler in einem Teilschritt propagieren sich bei Multi-Step unkontrolliert in nachfolgende Aufrufe.²¹³

²¹⁰ Vgl. Fathallah, Hernández, Staab 2025

²¹¹ Vgl. Fathallah, Hernández, Staab 2025, S. 5

²¹² `schulhoff_prompt_2024`

²¹³ `njifenjou_role_2024`

6.2.2 Token-Budget-Steuerung

Da lokale Modelle typischerweise über ein Kontextfenster von 32.768 Tokens verfügen und der System-Prompt sowie das Ausgabeformat einen fixen Anteil belegen, steht für die Findings ein variables Input-Budget zur Verfügung. Der Prompt-Builder implementiert eine Priorisierungsstrategie, die Findings nach Schweregrad sortiert und Einträge niedrigerer Priorität auslässt, sobald das Budget überschritten wird. Die Sortierung erfolgt auf Basis des normierten **severity**-Felds des UFS, was die quellenübergreifende Vergleichbarkeit der Schweregrade voraussetzt.

Darüber hinaus wird `grep` selektiv eingesetzt: Statt die gesamte Codebasis zu durchsuchen, werden ausschließlich Dateien analysiert, die durch `axe-core`- oder `Playwright`-Findings referenziert werden. Dieses Vorgehen begrenzt die Anzahl der Code-Snippets im Prompt und verhindert eine unkontrollierte Ausweitung des Token-Verbrauchs.

6.2.3 Modellwahl

Als Inferenzbackend wird Ollama eingesetzt, das die lokale Ausführung von Quantisierungsmodellen ohne GPU-Beschleunigung ermöglicht und damit die Datenschutzanforderungen der BITBW erfüllt (NFA-05).²¹⁴ Cloud-basierte Dienste wie die OpenAI API scheiden aus, da sie eine Übermittlung potenziell schützenswerter Quellcodeausschnitte an externe Server erforderten, was im Kontext öffentlicher Verwaltung nicht zulässig ist.

Für den Modellvergleich in der Evaluation (vgl. Kapitel 7) wurden drei Modelle ausgewählt: `qwen2.5-coder:7b` als ressourcenschonende Baseline mit Fokus auf Code-Aufgaben, `qwen3:latest` (ca. 8B Parameter) als Vertreter der neueren Modellgeneration mit Reasoning-Modus sowie `qwen3:32b` als qualitativ stärkste lokale Option innerhalb des verfügbaren Arbeitsspeichers von 96 GB. Die Modellgröße wird dabei bewusst als Variable im Evaluationsdesign behandelt, um den Trade-off zwischen Ausgabequalität und Laufzeit empirisch zu erfassen.

6.3 Ausgabeformat: entwicklernahe To-Do-Liste

Das Ausgabeformat des Systems ist als priorisierte To-Do-Liste konzipiert, die sich an Entwicklerinnen und Entwickler richtet und unmittelbar in den bestehenden Entwicklungsworkflow integrierbar sein soll. Die Wahl dieses Formats ergibt sich aus der Anforderungsanalyse (FA-05) sowie aus der Literatur, die zeigt, dass die Akzeptanz von Barrierefreiheitsempfehlungen stark von ihrer Handlungsorientierung abhängt.²¹⁵

²¹⁴paternăŝ _2025

²¹⁵Vgl. Fathallah, Hernández, Staab 2025, S. 8

Die Liste gliedert sich in zwei Prioritätsstufen: *Must-have*-Einträge bezeichnen Verstöße gegen Level-A- und Level-AA-Erfolgskriterien, die eine gesetzliche Konformitätspflicht begründen können, während *Nice-to-have*-Einträge Empfehlungen zur Verbesserung der Nutzungserfahrung umfassen. Jeder Eintrag enthält – sofern durch die grep-Anreicherung verfügbar – den Dateipfad der betroffenen Komponente, die Zeilennummer sowie einen konkreten Code-Fix-Vorschlag. Dieser Aufbau entspricht dem Qualitätskriterium der *Kontextsensitivität*: Das System geht über die reine Fehlerbenennung hinaus und liefert den entwicklungsrelevanten Kontext, der für eine unmittelbare Behebung erforderlich ist.

Es ist anzumerken, dass das Ausgabeformat durch einen System-Prompt vorgegeben wird, dessen Einhaltung durch das Sprachmodell nicht garantiert werden kann. Die Evaluation in Kapitel 7 zeigt, dass alle drei getesteten Modelle das vorgegebene Must-have/Nice-to-have-Schema in mindestens einem Lauf nicht einhalten. Diese Limitation wird in Abschnitt ?? diskutiert.

7 Implementierung des Proof of Concept

7.1 Technische Umsetzung

7.2 Beispielabläufe und Ergebnisse am Untersuchungsobjekt

8 Evaluation & Diskussion

Tab. 5: Beispielhafte Accessibility-Verstöße und zugehörige Prüfquellen

#	Violation	Kategorie	Erwartende Quelle	WCAG
1	<code></code> ohne alt-Attribut	syntaktisch	axe-core	1.1.1
2	<code><button></code> ohne sichtbaren Text und ohne aria-label	syntaktisch	axe-core	4.1.2
3	Formularfeld ohne <code><label></code>	syntaktisch	axe-core	1.3.1
4	Kontrastverhältnis $< 4.5 : 1$ (Text auf Hintergrund)	layout	axe-core	1.4.3
5	Fokus-Outline via CSS entfernt (<code>outline: none</code>)	layout	axe-core + Playwright	2.4.7
6	onClick-Handler ohne onKeyDown-Pendant	semantisch	grep	2.1.1
7	<code>role="button"</code> auf <code><div></code> ohne tabIndex	semantisch	axe-core + grep	4.1.2
8	Modaler Dialog ohne Fokus-Trap	semantisch	Playwright	2.1.2
9	Skip-Link fehlt	semantisch	Playwright	2.4.1
10	<code>aria-hidden="true"</code> auf fokussierbarem Element	syntaktisch	axe-core	4.1.2
11	Reihenfolge im DOM weicht von visueller Reihenfolge ab	semantisch	axe-core	1.3.2
12	Interaktives Element zu klein ($< 24 \times 24\text{px}$)	layout	grep (CSS-Pattern)	2.5.8

Beschreibung der Testkomponente – kurz erklären was sie ist und warum synthetisch Tabelle: eingebaute Violations (die obige Tabelle, gekürzt) Erkennungsrate – Tabelle + Recall/Precision-Werte Empfehlungsqualität – Tabelle mit Stufen + 2-3 konkrete Beispiele (ein Stufe-0, ein Stufe-2 gegenübergestellt) Satz zur Rechtfertigung warum quantitativ + qualitativ kombiniert wird BWEK Real-World als Zusatzvalidierung – 1 Absatz

8.1 Ergebnisse entlang der Kriterien und Limitationen

8.2 Handlungsempfehlungen und Ausblick

Tab. 6: Erkennungsrate der Violations im Proof of Concept

#	Violation	Erkannt (✓/×)	Quelle korrekt	False Positive	Anmerk
1	 ohne alt-Attribut	✓	axe-core ✓	–	
2	<button> ohne Label	✓	axe-core ✓	–	
3	Formularfeld ohne <label>				
4	Kontrastverhältnis < 4.5 : 1				
5	Fokus-Outline entfernt (outline: none)				
6	onClick ohne onKeyDown				
7	role="button" auf <div> ohne tabIndex				
8	Modal ohne Fokus-Trap				
9	Skip-Link fehlt				
10	aria-hidden="true" auf fokussierbarem Element				
11	DOM-Reihenfolge vs. visuelle Reihenfolge				
12	Interaktives Element zu klein (< 24 × 24px)				

Tab. 7: Bewertungsskala für die Empfehlungsqualität der LLM-Ausgaben

Stufe	Bezeichnung	Kriterium
2	Konkret	Enthält Dateiname, Zeilennummer und einen konkreten Fix-Vorschlag (z. B. <code>aria-label="..."</code>).
1	Teilweise	Enthält den richtigen Fix-Typ, jedoch ohne konkreten Codekontext (z. B. „Füge ein aria-label hinzu“).
0	Generisch	Beschreibt lediglich das Problem, ohne einen umsetzbaren Lösungsvorschlag.

Tab. 8: Bewertung der Empfehlungsqualität pro Violation

#	Violation	Stufe (0–2)	Anmerkung
1	 ohne alt-Attribut		
2	<button> ohne Label		
3	Formularfeld ohne <label>		
4	Kontrastverhältnis < 4.5 : 1		
5	Fokus-Outline entfernt (outline: none)		
6	onClick ohne onKeyDown		
7	role="button" auf <div> ohne tabIndex		
8	Modal ohne Fokus-Trap		
9	Skip-Link fehlt		
10	aria-hidden="true" auf fokussierbarem Element		
11	DOM-Reihenfolge vs. visuelle Reihenfolge		
12	Interaktives Element zu klein (< 24 × 24px)		

9 Fazit

Es funktioniert also tatsächlich, dass

Anhang

Anhangverzeichnis

Anhang 1	Komplexität von Home Pages	47
Anhang 2	Erweiterte Konzeptmatrix	47

Anhang 1: Komplexität von Home Pages

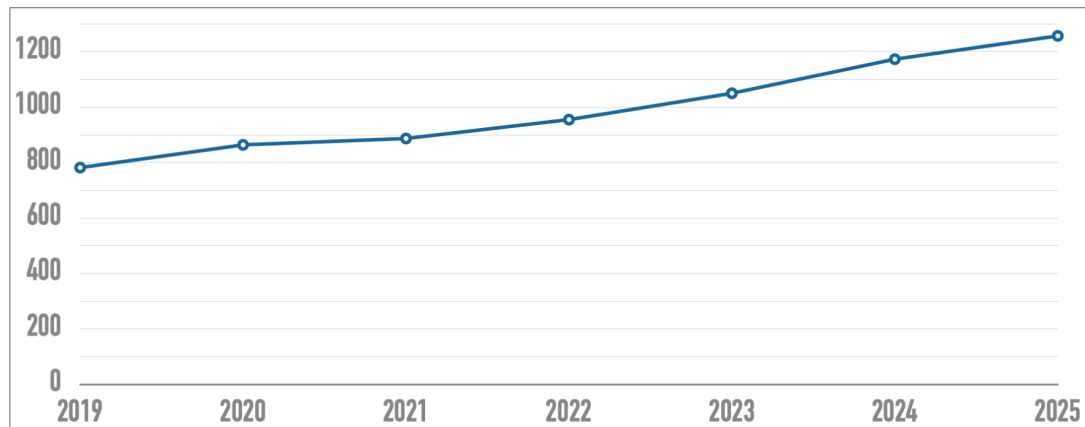


Abb. 6: Entwicklung der Komplexität von Home Pages in den letzten sechs Jahren. Die X-Achse zeigt die Jahre, die Y-Achse gibt die Anzahl der Elemente an.²¹⁶

Anhang 2: Erweiterte Konzeptmatrix

²¹⁶Entnommen aus WebAIM 2025

²¹⁷Vgl. Webster, Watson 2002.

Studie		Konzepte													
Autor(en) & Jahr	System	Ziel			Datenquellen			KI- / LLM-Ansatz			Evaluation			Kontext	
		WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA	Rechtl. Rahmen (BFSG etc.)
Hoch priorisierte Studien															
He et al. (2025)	GenA11y	x			x			x			x				
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x				
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x			x
Paternò et al. (2025)	TeVal	x		x	x			x	x			x			
Tafreshipour et al. (2024)	Ma11y	x			x	x					x				
Fernández-Navarro & Chicano (2025)	LLM Remediation	x	x	x	x		x	x			x			x	
Mittel priorisierte Studien															
Yu et al. (2025)	GenAI Screen Reader	x	x		x		x	x	x		x	x	x		
Huang et al. (2024)	ACCESS	x	x	x	x	x		x			x				
Pool (2023)	Testaro	x			x	x					x				
Mowar et al. (2024)	Copilot Study	x		x				x					x		
Abu Doush & Kassem (2024)	AI Code Study	x	x	x				x			x				
Guriță & Vatavu (2025)	AI UI Study	x						x			x				
Kodithuwakku & Wick. (2025)	Tool Eval.	x			x						x				
Manca et al. (2023)	Transparency	x		x	x							x	x		
Jiang & Nam (2025)	Cursor Rules			x			x	x				x			
Gubbi Mohanbabu & Pavel (2024)	Context-Aware Img.	x	x					x	x		x	x	x		
Campoverde-Molina & Luján-Mora (2025)	AI A11y SMS	x	x	x	x			x				x			
Leedy et al. (2025)	GenAI Builder Eval.	x			x			x			x	x			
Patel & Melcer (2025)	AIDA Study	x		x				x				x	x		
Aljedaani et al. (2024)	ChatGPT A11y Code	x	x	x			x	x			x				
Niedrig priorisierte Studien															
Abou-Zahra et al. (2018)	AI for A11y	x						x				x			x
Delnevo et al. (2024)	LLM Interaction	x	x					x				x		x	
Draffan et al. (2020)	Web2Access+AI	x			x			x				x			
Eigener Proof of Concept															
Martínez Campo (2026)	ACE (angestrebt)	x	x	x	x	x	x	x	x	x		x		x	x

Tab. 9: Konzeptmatrix nach Webster Watson²¹⁷: Vollständige Übersicht aller analysierten Studien im Vergleich zum eigenen Proof of Concept (PoC). **x** = Konzept vorhanden/adressiert; leer = nicht adressiert.

Literaturverzeichnis

- Abou-Zahra, Shadi; Brewer, Judy; Cooper, Michael (2018):** Artificial Intelligence (AI) for Web Accessibility: Is Conformance Evaluation a Way Forward? In: *Proceedings of the 15th International Web for All Conference*. W4A '18. New York, NY, USA: Association for Computing Machinery, S. 1–4. ISBN: 978-1-4503-5651-0. DOI: 10.1145/3192714.3192834. URL: <https://dl.acm.org/doi/10.1145/3192714.3192834> (Abruf: 24.02.2026).
- Abuaddous, Hayfa Y.; Jali, Mohd Zalisham; Basir, Nurlida (2016):** Web Accessibility Challenges. In: *International Journal of Advanced Computer Science and Applications (ijacsa)* 7.10. ISSN: 2156-5570. DOI: 10.14569/IJACSA.2016.071023. URL: <https://thesai.org/Publications/ViewPaper?Volume=7&Issue=10&Code=ijacsa&SerialNo=23> (Abruf: 04.03.2026).
- Anthropic (2026a):** Claude Model Overview. URL: <https://www.anthropic.com/claude> (Abruf: 01.03.2026).
- Anthropic (2026b):** System Card: Claude Sonnet 4.6. URL: <https://www-cdn.anthropic.com/78073f739564e986ff3e28522761a7a0b4484f84.pdf> (Abruf: 02.03.2026).
- Anthropic (2026c):** Resources. URL: <https://platform.claude.com/docs/en/about-claude/glossary> (Abruf: 01.03.2026).
- Bai, Aiden (2022):** A Fast Compiler-Augmented Virtual DOM for the Web. In: *arXiv*. URL: <https://arxiv.org/abs/2202.08409>.
- Baskerville, Richard; Myers, Michael D. (2004):** Special Issue on Action Research in Information Systems: Making IS Research Relevant to Practice Foreword. In: *MIS Quarterly* 28.3, S. 329–335.
- Baskerville, Richard L. (1999):** Investigating Information Systems with Action Research. In: *Communications of the Association for Information Systems* 2.19, S. 1–32.
- Bassi, Barry; Delnevo, Giovanni; Franco, Mirko; Gaggi, Ombretta; Gatto, Salvatore; Mirri, Silvia; Olaiya, Kelvin (2025):** An Assessment of LLM-Based Auditing and Validation for Web Accessibility. In: *Proceedings of the 2025 International Conference on Information Technology for Social Good*. GoodIT '25. New York, NY, USA: Association for Computing Machinery, S. 297–305. ISBN: 979-8-4007-2089-5. DOI: 10.1145/3748699.3749805. URL: <https://dl.acm.org/doi/10.1145/3748699.3749805> (Abruf: 24.02.2026).
- Bender, Emily M.; Gebru, Timnit; McMillan-Major, Angelina; Shmitchell, Shmargaret (2021):** On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. FAccT '21. New York, NY, USA: Association for Computing Machinery, S. 610–623. ISBN: 978-1-4503-8309-7. DOI: 10.1145/3442188.3445922. URL: <https://dl.acm.org/doi/10.1145/3442188.3445922> (Abruf: 05.03.2026).
- BITBW (2021):** 6 Jahre BITBW. URL: <https://bitbw.de/neuigkeiten/artikel/6-jahre-bitbw-1> (Abruf: 28.02.2026).
- Brown, Tom B.; Mann, Benjamin; Ryder, Nick; Subbiah, Melanie; Kaplan, Jared D.; Dhariwal, Prafulla; Neelakantan, Arvind; Shyam, Pranav; Sastry, Girish; Askell, Amanda; Agarwal, Sandhini; Herbert-Voss, Ariel; Krueger, Gretchen; Henighan,**

- Tom; Child, Rewon; Ramesh, Aditya; Ziegler, Daniel M.; Wu, Jeffrey; Winter, Clemens; Hesse, Christopher; Chen, Mark; Sigler, Eric; Litwin, Mateusz; Gray, Scott; Chess, Benjamin; Clark, Jack; Berner, Christopher; McCandlish, Sam; Radford, Alec; Sutskever, Ilya; Amodei, Dario (2020):** Language Models are Few-Shot Learners. Definition von zero-/one-/few-shot (in-context learning) auf S. 4. arXiv: 2005.14165 [cs.CL]. URL: <https://arxiv.org/abs/2005.14165> (Abruf: 28.02.2026).
- Brynjolfsson, Erik; Li, Danielle; Raymond, Lindsey R (2023):** Generative AI at Work. In.
- Bundesamt der Justiz und für Verbraucherschutz (2026):** Gesetz zur Umsetzung der Richtlinie (EU) 2019/882 des Europäischen Parlaments und des Rates über die Barrierefreiheitsanforderungen für Produkte und Dienstleistungen 1. URL: <https://www.gesetze-im-internet.de/bfsg/> (Abruf: 24.02.2026).
- Bundesministerium für Digitales und Staatsmodernisierung (2025):** Barrierefreie-Informationstechnik-Verordnung (BITV 2.0). URL: <https://www.barrierefreiheit-dienstekonsolidierung.bund.de/Web/PB/DE/gesetze-und-richtlinien/bitv2-0/bitv2-0-node.html> (Abruf: 24.02.2026).
- Chiou, Paul T.; Alotaibi, Ali S.; Halfond, William G. J. (2021):** Detecting and localizing keyboard accessibility failures in web applications. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, S. 855–867. ISBN: 978-1-4503-8562-6. DOI: 10.1145/3468264.3468581. URL: <https://dl.acm.org/doi/10.1145/3468264.3468581> (Abruf: 17.03.2026).
- Delnevo, Giovanni; Andruccioli, Manuel; Mirri, Silvia (2024):** On the Interaction with Large Language Models for Web Accessibility: Implications and Challenges. In: *2024 IEEE 21st Consumer Communications & Networking Conference (CCNC)*. 2024 IEEE 21st Consumer Communications & Networking Conference (CCNC). ISSN: 2331-9860, S. 1–6. DOI: 10.1109/CCNC51664.2024.10454680. URL: <https://ieeexplore.ieee.org/document/10454680> (Abruf: 24.02.2026).
- Deque Systems (2026):** *dequelabs/axe-core*. original-date: 2015-06-10T15:26:45Z. URL: <https://github.com/dequelabs/axe-core> (Abruf: 24.02.2026).
- Deutscher Bundestag (2026):** BFSG (Barrierefreiheitsstärkungsgesetz). URL: <https://bfsg-gesetz.de/> (Abruf: 24.02.2026).
- Doush, Iyad Abu; Kassem, Reem (2025):** Evaluating AI-Generated Web Code for Accessibility Compliance: A Metric-Driven Approach. In: *Proceedings of the 11th International Conference on Software Development and Technologies for Enhancing Accessibility and Fighting Info-exclusion*. DSAI '24. New York, NY, USA: Association for Computing Machinery, S. 338–344. ISBN: 979-8-4007-0729-2. DOI: 10.1145/3696593.3696614. URL: <https://dl.acm.org/doi/10.1145/3696593.3696614> (Abruf: 24.02.2026).
- Europäisches Parlament und Rat der Europäischen Union (2016):** *Verordnung (EU) 2016/679 des Europäischen Parlaments und des Rates vom 27. April 2016 zum Schutz natürlicher Personen bei der Verarbeitung personenbezogener Daten, zum freien Datenverkehr und*

- zur Aufhebung der Richtlinie 95/46/EG (Datenschutz-Grundverordnung). URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (Abruf: 10.03.2026).
- European Commission (2026)**: European accessibility act (EAA). URL: https://commission.europa.eu/strategy-and-policy/policies/justice-and-fundamental-rights/disability/european-accessibility-act-eaa_en (Abruf: 24.02.2026).
- Fathallah, Nadeen; Hernández, Daniel; Staab, Steffen (2025)**: AccessGuru: Leveraging LLMs to Detect and Correct Web Accessibility Violations in HTML Code. In: *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '25. New York, NY, USA: Association for Computing Machinery, S. 1–22. ISBN: 979-8-4007-0676-9. DOI: 10.1145/3663547.3746360. URL: <https://dl.acm.org/doi/10.1145/3663547.3746360> (Abruf: 24.02.2026).
- Fernandes, Nádia; Costa, Daniel; Duarte, Carlos; Carriço, Luís (2012)**: Evaluating the Accessibility of Web Applications. In: *Procedia Computer Science*. Proceedings of the 4th International Conference on Software Development for Enhancing Accessibility and Fighting Info-exclusion (DSAI 2012) 14, S. 28–35. ISSN: 1877-0509. DOI: 10.1016/j.procs.2012.10.004. URL: <https://www.sciencedirect.com/science/article/pii/S1877050912007661> (Abruf: 05.03.2026).
- Flanagan, David (2020)**: JavaScript: The definitive guide. O'Reilly. ISBN: 0596805527.
- Gao, Yunfan; Xiong, Yun; Gao, Xinyu; Jia, Kang; Pan, Jinliang et al. (2023)**: Retrieval-Augmented Generation for Large Language Models: A Survey. In: *arXiv preprint arXiv:2312.10997*.
- Google (2025)**: Lighthouse : Chrome for developers. URL: <https://developer.chrome.com/docs/lighthouse/overview> (Abruf: 28.02.2026).
- Gubbi Mohanbabu, Ananya; Pavel, Amy (2024)**: Context-Aware Image Descriptions for Web Accessibility. In: *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '24. New York, NY, USA: Association for Computing Machinery, S. 1–17. ISBN: 979-8-4007-0677-6. DOI: 10.1145/3663548.3675658. URL: <https://dl.acm.org/doi/10.1145/3663548.3675658> (Abruf: 24.02.2026).
- Guriță, Alexandra-Elena; Vatavu, Radu-Daniel (2025)**: Insights and Implications of Evaluating Accessibility Compliance in AI-Generated Web Interfaces. In: *Companion Proceedings of the ACM on Web Conference 2025*. WWW '25. New York, NY, USA: Association for Computing Machinery, S. 996–1000. ISBN: 979-8-4007-1331-6. DOI: 10.1145/3701716.3715552. URL: <https://dl.acm.org/doi/10.1145/3701716.3715552> (Abruf: 24.02.2026).
- Harrenstien, Ken (2009)**: Automatic captions in YouTube. Official Google Blog. URL: <https://googleblog.blogspot.com/2009/11/automatic-captions-in-youtube.html> (Abruf: 04.03.2026).
- He, Ziyao; Huq, Syed Fatiul; Malek, Sam (2025)**: Enhancing Web Accessibility: Automated Detection of Issues with Generative AI. In: *Proc. ACM Softw. Eng.* 2 (FSE), FSE101:2264–FSE101:2287. DOI: 10.1145/3729371. URL: <https://dl.acm.org/doi/10.1145/3729371> (Abruf: 24.02.2026).
- Heinemann, Daniela (2024)**: „Barrierefreiheit“. In: *Digitalisierte Verwaltung - Vernetztes E-Government*. Hrsg. von Margrit Seckelmann. Berlin: Erich Schmidt Verlag GmbH & Co. KG,

- S. 469–488. ISBN: 978-3-503-23763-0. DOI: 10.37307/b.978-3-503-23763-0.15. URL: <https://doi.org/10.37307/b.978-3-503-23763-0.15> (Abruf: 24.02.2026).
- Hevner, Alan R.; March, Salvatore T.; Park, Jinsoo; Ram, Sudha (2004):** Design Science in Information Systems Research. In: *MIS Quarterly* 28.1, S. 75–105.
- Huang, Calista; Ma, Alyssa; Vyasamudri, Suchir; Puype, Eugenie; Kamal, Sayem; Garcia, Juan Belza; Cheema, Salar; Lutz, Michael (2024):** ACCESS: Prompt Engineering for Automated Web Accessibility Violation Corrections. DOI: 10.48550/arXiv.2401.16450. arXiv: 2401.16450[cs]. URL: <http://arxiv.org/abs/2401.16450> (Abruf: 25.02.2026).
- Huang, Lei; Yu, Weijiang; Ma, Weitao; Zhong, Weihong; Feng, Zhangyin; Wang, Haotian; Chen, Qianglong; Peng, Weihua; Feng, Xiaocheng; Qin, Bing; Liu, Ting (2025):** A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. In: *ACM Transactions on Information Systems* 43.2, S. 1–55. ISSN: 1046-8188, 1558-2868. DOI: 10.1145/3703155. arXiv: 2311.05232[cs]. URL: <http://arxiv.org/abs/2311.05232> (Abruf: 05.03.2026).
- Hui, Binyuan; Yang, Jian; Cui, Zeyu; Yang, Jiayi; Liu, Dayiheng; Zhang, Lei; Liu, Tianyu; Zhang, Jiajun; Yu, Bowen; Lu, Keming; Dang, Kai; Fan, Yang; Zhang, Yichang; Yang, An; Men, Rui; Huang, Fei; Zheng, Bo; Miao, Yibo; Quan, Shanghaoran; Feng, Yunlong; Ren, Xingzhang; Ren, Xuancheng; Zhou, Jingren; Lin, Junyang (2024):** Qwen2.5-Coder Technical Report. DOI: 10.48550/arXiv.2409.12186. arXiv: 2409.12186[cs]. URL: <http://arxiv.org/abs/2409.12186> (Abruf: 15.03.2026).
- Kerkmann, Friederike; Lewandowski, Dirk (2015):** Barrierefreie Informationssysteme: Zugänglichkeit für Menschen mit Behinderung in Theorie und Praxis. Google-Books-ID: T5ylCQA-AQBAJ. Walter de Gruyter GmbH & Co KG. 292 S. ISBN: 978-3-11-033729-7.
- Kodithuwakku, Tashmi; Wickramaarachchi, Dilani (2025):** Assessing the Limits of Automated Accessibility Testing: Insights for QA Engineers and Tool Developers. In: *2025 5th International Conference on Advanced Research in Computing (ICARC)*. 2025 5th International Conference on Advanced Research in Computing (ICARC), S. 1–6. DOI: 10.1109/ICARC64760.2025.10963173. URL: <https://ieeexplore.ieee.org/document/10963173> (Abruf: 24.02.2026).
- Kouroupetroglou, Georgios (2013):** Assistive Technologies and Computer Access for Motor Disabilities. IGI Global Scientific Publishing. ISBN: 978-1-4666-4438-0. URL: <https://www.igi-global.com/book/assistive-technologies-computer-access-motor/75832>.
- Krok, Ewa (2024):** Digital Accessibility as an Essential Element of an Inclusive Organization. In: *European Research Studies XXVII (Special A)*, S. 857–876. ISSN: 1108-2976. URL: <https://ersj.eu/journal/3752> (Abruf: 17.03.2026).
- Krotoff, Tanguy (2026):** Front-end frameworks popularity (React, Vue, Angular and Svelte). Gist. URL: <https://gist.github.com/tkrotoff/b1caa4c3a185629299ec234d2314e190> (Abruf: 05.03.2026).
- Land Baden-Württemberg (2025):** Neuer BW-Empfangsclient treibt Digitalisierung der Verwaltung voran. URL: <https://www.baden-wuerttemberg.de/de/service/presse/pressemitteilung>

- tteilung/pid/neuer-bw-empfangsclient-treibt-digitalisierung-der-verwaltung-voran-1 (Abruf: 24.02.2026).
- Landesrecht BW (2015)**: Gesetz zur Errichtung der Landesoberbehörde BITBW. URL: <https://www.landesrecht-bw.de/bsbw/document/jlr-ITErGBWV3P2> (Abruf: 28.02.2026).
- Lazar, Jonathan; Feng, Jinjuan Heidi; Hochheiser, Harry (2017)**: Research Methods in Human-Computer Interaction. Elsevier. ISBN: 978-0-12-805390-4.
- Lewis, Patrick; Perez, Ethan; Piktus, Aleksandra; Petroni, Fabio; Karpukhin, Vladimir; Goyal, Naman; Küttler, Heinrich; Lewis, Mike; Yih, Wen-tau; Rocktäschel, Tim; Riedel, Sebastian; Kiela, Douwe (2021)**: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. DOI: 10.48550/arXiv.2005.11401. arXiv: 2005.11401[cs]. URL: <http://arxiv.org/abs/2005.11401> (Abruf: 05.03.2026).
- Manca, Marco; Palumbo, Vanessa; Paternò, Fabio; Santoro, Carmen (2023)**: The Transparency of Automatic Web Accessibility Evaluation Tools: Design Criteria, State of the Art, and User Perception. In: *ACM Trans. Access. Comput.* 16.1, 3:1–3:36. ISSN: 1936-7228. DOI: 10.1145/3556979. URL: <https://dl.acm.org/doi/10.1145/3556979> (Abruf: 24.02.2026).
- Meta Open Source (2026a)**: React. URL: <https://react.dev/> (Abruf: 05.03.2026).
- Meta Open Source (2026b)**: Render and Commit. URL: <https://react.dev/learn/render-and-commit> (Abruf: 05.03.2026).
- Microsoft (2026)**: Installation | Playwright. URL: <https://playwright.dev/docs/intro> (Abruf: 24.02.2026).
- Morris, Meredith Ringel; Zolyomi, Annuska; Yao, Catherine; Bahram, Sina; Bigham, Jeffrey P.; Kane, Shaun K. (2016)**: "With most of it being pictures now, I rarely use it: Understanding Twitter's Evolving Accessibility to Blind Users. In: *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. CHI '16. New York, NY, USA: Association for Computing Machinery, S. 5506–5516. ISBN: 978-1-4503-3362-7. DOI: 10.1145/2858036.2858116. URL: <https://dl.acm.org/doi/10.1145/2858036.2858116> (Abruf: 04.03.2026).
- Morrison, Cecily; Cutrell, Edward; Dhareshwar, Anupama; Doherty, Kevin; Thieme, Anja; Taylor, Alex (2017)**: Imagining Artificial Intelligence Applications with People with Visual Disabilities using Tactile Ideation. In: *Proceedings of the 19th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '17. New York, NY, USA: Association for Computing Machinery, S. 81–90. ISBN: 978-1-4503-4926-0. DOI: 10.1145/3132525.3132530. URL: <https://dl.acm.org/doi/10.1145/3132525.3132530> (Abruf: 04.03.2026).
- Mowar, Peya (2024)**: Accessibility in AI-Assisted Web Development. In: *Proceedings of the 21st International Web for All Conference*. W4A '24. New York, NY, USA: Association for Computing Machinery, S. 123–125. ISBN: 979-8-4007-1030-8. DOI: 10.1145/3677846.3679054. URL: <https://dl.acm.org/doi/10.1145/3677846.3679054> (Abruf: 24.02.2026).
- Njifenjou, Ahmed; Sucal, Virgile; Jabaian, Bassam; Lefèvre, Fabrice (2024)**: Role-Play Zero-Shot Prompting with Large Language Models for Open-Domain Human-Machine

- Conversation. DOI: 10.48550/arXiv.2406.18460. arXiv: 2406.18460[cs]. URL: <http://arxiv.org/abs/2406.18460> (Abruf: 28.02.2026).
- Ollama (2026)**: Quickstart. URL: <https://docs.ollama.com/quickstart> (Abruf: 04.03.2026).
- OpenAI (2023)**: GPT-4 Technical Report. In: *arXiv preprint arXiv:2303.08774*. arXiv: 2303.08774. URL: <https://arxiv.org/abs/2303.08774> (Abruf: 01.03.2026).
- OpenAI (2026)**: Create completion. URL: <https://developers.openai.com/api/reference/resources/completions/methods/create/> (Abruf: 01.03.2026).
- Organization, World Health (2026)**: World Report on Disability. WHO Library Cataloguing-in-Publication Data. NLM classification: HV 1553. World Health Organization. ISBN: 978-92-4-156418-2; 978-92-4-068800-1 (PDF); 978-92-4-068636-6 (ePUB); 978-92-4-068637-3 (Daisy); 978-92-4-068521-5 (PDF, original). URL: <https://www.who.int/teams/noncommunicable-diseases/sensory-functions-disability-and-rehabilitation/world-report-on-disability> (Abruf: 04.03.2026).
- Paternò, Fabio; Vinci, Manuela; Manca, Marco; Iannuzzi, Nicola (2025)**: How an LLM Can Improve Automatic Web Accessibility Validation ? In: *Proceedings of the 16th Biannual Conference of the Italian SIGCHI Chapter*. CHIItaly '25. New York, NY, USA: Association for Computing Machinery, S. 1–8. ISBN: 979-8-4007-2102-1. DOI: 10.1145/3750069.3750310. URL: <https://dl.acm.org/doi/10.1145/3750069.3750310> (Abruf: 24.02.2026).
- Peffer, Ken; Tuunanen, Tuure; Rothenberger, Marcus A.; Chatterjee, Samir (2007)**: A Design Science Research Methodology for Information Systems Research. In: *Journal of Management Information Systems* 24.3, S. 45–77. DOI: 10.2753/MIS0742-1222240302.
- Pérez, J. Eduardo; Arrue, Myriam; Valencia, Xabier; Abascal, Julio (2020)**: Longitudinal Study of Two Virtual Cursors for People With Motor Impairments: A Performance and Satisfaction Analysis on Web Navigation. In: *IEEE Access*. DOI: 10.1109/ACCESS.2020.3001766.
- Pool, Jonathan Robert (2023)**: Testaro: Efficient Ensemble Testing for Web Accessibility. In: *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '23. New York, NY, USA: Association for Computing Machinery, S. 1–4. ISBN: 979-8-4007-0220-4. DOI: 10.1145/3597638.3614505. URL: <https://dl.acm.org/doi/10.1145/3597638.3614505> (Abruf: 24.02.2026).
- Purwandari, Nuraini; Dewi, Ratna Kusuma; Rinaldo; Sucipto, Purwo Agus (2025)**: Policy in Practice: A Systematic Review of WCAG 2.2 and ADA 2024 Effects on Web and Mobile Accessibility. In: *Digitus*. DOI: 10.61978/digitus.v3i2.957. URL: https://www.researchgate.net/publication/396366120_Policy_in_Practice_A_Systematic_Review_of_WCAG_22_and_ADA_2024_Effects_on_Web_and_Mobile_Accessibility (Abruf: 04.03.2026).
- Robie, Jonathan (2026)**: What is the Document Object Model? URL: <https://www.w3.org/TR/WD-DOM/introduction.html> (Abruf: 04.03.2026).
- Russell, Stuart; Norvig, Peter (2021)**: Artificial Intelligence: A Modern Approach. Hoboken: Pearson. 1136 S. ISBN: 978-0-13-461099-3.

- Schulhoff, Sander; Ilie, Michael; Balepur, Nishant; Kahadze, Konstantine; Liu, Amanda; Si, Chenglei; Li, Yinheng; Gupta, Aayush; Han, HyoJung; Schulhoff, Sevin; Dulepet, Pranav Sandeep; Vidyadhara, Saurav; Ki, Dayeon; Agrawal, Sweta; Pham, Chau; Kroiz, Gerson; Li, Feileen; Tao, Hudson; Srivastava, Ashay; Da Costa, Hevander; Gupta, Saloni; Rogers, Megan L.; Goncarenco, Inna; Sarli, Giuseppe; Galynker, Igor; Peskoff, Denis; Carpuat, Marine; White, Jules; Anadkat, Shyamal; Hoyle, Alexander; Resnik, Philip (2024): The Prompt Report: A Systematic Survey of Prompting Techniques. In: *arXiv preprint arXiv:2406.06608*. arXiv: 2406.06608. URL: <https://arxiv.org/abs/2406.06608> (Abruf: 09.03.2026).
- Silvestre, Pedro F.; Pietzuch, Peter (2025): Systems Opportunities for LLM Fine-Tuning using Reinforcement Learning. In: *Proceedings of the 5th Workshop on Machine Learning and Systems*. EuroMLSys '25. New York, NY, USA: Association for Computing Machinery, S. 90–99. ISBN: 979-8-4007-1538-9. DOI: 10.1145/3721146.3721944. URL: <https://dl.acm.org/doi/10.1145/3721146.3721944> (Abruf: 09.03.2026).
- Śmite, Darja; Wohlin, Claes; Galviņa, Zane; Prikladnicki, Rafael (2012): An empirically based terminology and taxonomy for Global Software Engineering. In: *Empirical Software Engineering* 19.1, S. 105–153. DOI: 10.1007/s10664-012-9217-9.
- Sommerville, Ian (2016): Software Engineering. 10. Aufl. Pearson. ISBN: 978-0133943030.
- Souza, Aline; de Souza Filho, José Cezar; Bezerra, Carla; Alves, Victor Anthony; Lima, Lara; Marques, Anna Beatriz; Teixeira Monteiro, Ingrid (2024): Accessibility Evaluation of Web Systems for People with Visual Impairments: Findings from a Literature Survey. In: *Proceedings of the XXIII Brazilian Symposium on Human Factors in Computing Systems*. IHC '24. New York, NY, USA: Association for Computing Machinery, S. 1–13. ISBN: 979-8-4007-1224-1. DOI: 10.1145/3702038.3702090. URL: <https://dl.acm.org/doi/10.1145/3702038.3702090> (Abruf: 04.03.2026).
- Stack Overflow (2025): Survey index. URL: <https://survey.stackoverflow.co/2025/> (Abruf: 05.03.2026).
- Tafreshipour, Mahan; Deshpande, Anmol; Mehralian, Forough; Ahmed, Iftexhar; Malek, Sam (2024): Mally: A Mutation Framework for Web Accessibility Testing. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, S. 100–111. ISBN: 979-8-4007-0612-7. DOI: 10.1145/3650212.3652113. URL: <https://dl.acm.org/doi/10.1145/3650212.3652113> (Abruf: 24.02.2026).
- Touvron, Hugo; Martin, Louis; Stone, Kevin; Albert, Peter; Almahairi, Amjad; Babaei, Yasmine; Bashlykov, Nikolay; Batra, Soumya; Bhargava, Prajjwal; Bhosale, Shruti; Bikel, Dan; Blecher, Lukas; Ferrer, Cristian Canton; Chen, Moya; Cucurull, Guillem; Esiobu, David; Fernandes, Jude; Fu, Jeremy; Fu, Wenying; Fuller, Brian; Gao, Cynthia; Goswami, Vedanuj; Goyal, Naman; Hartshorn, Anthony; Hosseini, Saghar; Hou, Rui; Inan, Hakan; Kardas, Marcin; Kerkez, Viktor; Khabsa, Madian; Kloumann, Isabel; Korenev, Artem; Koura, Punit Singh; Lachaux, Marie-Anne; Lavril, Thibaut; Lee, Jenya; Liskovich, Diana; Lu, Yinghai; Mao, Yuning; Martinet,

- Xavier; Mihaylov, Todor; Mishra, Pushkar; Molybog, Igor; Nie, Yixin; Poulton, Andrew; Reizenstein, Jeremy; Rungta, Rashi; Saladi, Kalyan; Schelten, Alan; Silva, Ruan; Smith, Eric Michael; Subramanian, Ranjan; Tan, Xiaoqing Ellen; Tang, Binh; Taylor, Ross; Williams, Adina; Kuan, Jian Xiang; Xu, Puxin; Yan, Zheng; Zarov, Iliyan; Zhang, Yuchen; Fan, Angela; Kambadur, Melanie; Narang, Sharan; Rodriguez, Aurelien; Stojnic, Robert; Edunov, Sergey; Scialom, Thomas (2023): Llama 2: Open Foundation and Fine-Tuned Chat Models. In: *arXiv preprint arXiv:2307.09288*. arXiv: 2307.09288 [cs.CL]. URL: <https://arxiv.org/abs/2307.09288> (Abruf: 01.03.2026).
- United Nations (2006a): Convention on the Rights of Persons with Disabilities. URL: <https://www.ohchr.org/en/instruments-mechanisms/instruments/convention-rights-persons-disabilities> (Abruf: 28.02.2026).
- United Nations (2006b): Convention on the Rights of Persons with Disabilities. United Nations. URL: <https://www.un.org/disabilities/documents/convention/convoptprot-e.pdf> (Abruf: 04.03.2026).
- Wang, Yuqing; Zhao, Yun (2024): Metacognitive Prompting Improves Understanding in Large Language Models. DOI: 10.48550/arXiv.2308.05342. arXiv: 2308.05342[cs]. URL: <http://arxiv.org/abs/2308.05342> (Abruf: 09.03.2026).
- WebAIM (2019): The WebAIM Million - 2019 report on the accessibility of the top 1,000,000 home pages. URL: <https://webaim.org/projects/million/2019> (Abruf: 24.02.2026).
- WebAIM (2025): The WebAIM Million - 2025 report on the accessibility of the top 1,000,000 home pages. URL: <https://webaim.org/projects/million/> (Abruf: 24.02.2026).
- WebAIM (2026a): Wave Web Accessibility Evaluation Tools. URL: <https://wave.webaim.org/> (Abruf: 28.02.2026).
- WebAIM (2026b): WebAIM: Keyboard Accessibility. URL: <https://webaim.org/techniques/keyboard/> (Abruf: 04.03.2026).
- Webster, Jane; Watson, Richard T. (2002): Analyzing the Past to Prepare for the Future: Writing a Literature Review. In: *MIS Quarterly* 26.2, S. 2–6.
- Wei, Jason; Tay, Yi; Bommasani, Rishi; Raffel, Colin; Zoph, Barret; Borgeaud, Sebastian; Yogatama, Dani; Bosma, Maarten; Zhou, Denny; Metzler, Donald; Chi, Ed H.; Hashimoto, Tatsunori; Vinyals, Oriol; Liang, Percy; Dean, Jeffrey; Fedus, William (2022): Emergent Abilities of Large Language Models. In: *Transactions on Machine Learning Research*, S. 1–35. URL: <https://arxiv.org/abs/2206.07682> (Abruf: 28.02.2026).
- Wei, Jason; Wang, Xuezhi; Schuurmans, Dale; Bosma, Maarten; Ichter, Brian; Xia, Fei; Chi, Ed; Le, Quoc V.; Zhou, Denny (2022): Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. Definition von Chain-of-Thought Prompting auf S. 2. arXiv: 2201.11903 [cs.CL].
- World Health Organization (2026): Disability. URL: https://www.who.int/health-topics/disability#tab=tab_1 (Abruf: 28.02.2026).
- World Wide Web Consortium (W3C) (2018): Web Content Accessibility Guidelines (WCAG) 2.1. URL: <https://www.w3.org/TR/WCAG21/> (Abruf: 24.02.2026).

- World Wide Web Consortium (W3C) (2023)**: Web Content Accessibility Guidelines (WCAG) 2.2. URL: <https://www.w3.org/TR/WCAG22/> (Abruf: 24.02.2026).
- World Wide Web Consortium (W3C) (2026a)**: Evaluation-Tool-List. URL: <https://www.w3.org/WAI/test-evaluate/tools/list/> (Abruf: 28.02.2026).
- World Wide Web Consortium (W3C) (2026b)**: Web Accessibility Initiative. URL: <https://www.w3.org/WAI/> (Abruf: 28.02.2026).
- Yao, Shunyu; Zhao, Jeffrey; Yu, Dian; Du, Nan; Shafran, Izhak; Narasimhan, Karthik; Cao, Yuan (2023)**: ReAct: Synergizing Reasoning and Acting in Language Models. Definition/Intuition von ReAct (interleaved reasoning+acting) auf S. 2; publiziert bei ICLR 2023. arXiv: 2210.03629 [cs.CL]. URL: <https://arxiv.org/pdf/2210.03629> (Abruf: 28.02.2026).
- Yin, Robert K. (2018)**: Case Study Research and Applications: Design and Methods. Los Angeles London New Delhi Singapore Washington DC Melbourne: SAGE Publications, Inc. 352 S. ISBN: 978-1-5063-3616-9.

Erklärung zur Verwendung generativer KI-Systeme

Bei der Erstellung der eingereichten Arbeit habe ich auf künstlicher Intelligenz (KI) basierte Systeme benutzt:

☒ ja

☐ nein²¹⁸

Falls ja: Die nachfolgend aufgeführten auf künstlicher Intelligenz (KI) basierten Systeme habe ich bei der Erstellung der eingereichten Arbeit benutzt:

- 1.
- 2.
3. ...

Ich erkläre, dass ich

- mich aktiv über die Leistungsfähigkeit und Beschränkungen der oben genannten KI-Systeme informiert habe,²¹⁹
- die aus den oben angegebenen KI-Systemen direkt oder sinngemäß übernommenen Passagen gekennzeichnet habe,
- überprüft habe, dass die mithilfe der oben genannten KI-Systeme generierten und von mir übernommenen Inhalte faktisch richtig sind,
- mir bewusst bin, dass ich als Autorin bzw. Autor dieser Arbeit die Verantwortung für die in ihr gemachten Angaben und Aussagen trage.

Die oben genannten KI-Systeme habe ich wie im Folgenden dargestellt eingesetzt:

Arbeitsschritt in der wissenschaftlichen Arbeit	Eingesetzte(s) KI-System(e)	Beschreibung der Verwendungsweise

²¹⁸Die Erklärung ist in jedem Fall zu unterzeichnen, auch wenn Sie keine KI-Systeme genutzt haben und Ihr Kreuz bei „nein“ gesetzt haben.

²¹⁹U.a. gilt es hierbei zu beachten, dass an KI weitergegebene Inhalte ggf. als Trainingsdaten genutzt und wiederverwendet werden. Dies ist insb. für betriebliche Aspekte als kritisch einzustufen.

(Ort, Datum)

(Unterschrift)

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema: *Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse von Rich-Client-Webanwendungen mittels Tool-Reports, Playwright-Interaktionen und Codeanalyse* selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

(Ort, Datum)

(Unterschrift)